

Diploma Project

**Hybrid Quantum-Classical Machine
Learning for Autoencoder-Based
Wireless Communication Systems**

Ireneos Constantinou



University of Cyprus
Department of Electrical and
Computer Engineering

Department of Electrical and Computer Engineering

May 2026

UNIVERSITY OF CYPRUS

Faculty of Engineering

Department of Electrical and Computer Engineering

Hybrid Quantum-Classical Machine Learning for Autoencoder-Based Wireless Communication Sys- tems

Ireneos Constantinou

Advisor: Professor

Dr Ioannis Krikides

May 2026

Acknowledgements

Four years have passed since I joined the University of Cyprus. Many people came and went in my life, and strong relationships have been formed relationships bonded by deep aspirations about life, goals, and friendships through both fun and difficult times. I leave this university eager to achieve everything. Although my opinions regarding the academic side of this journey could be defined as controversial, I am thankful for all the opportunities and the fields of engineering, whereas the engineering department of the university has introduced me through the courses and the activities. I want to also personally thank Professor Dr. Ioannis Krikidis, who introduced me to the field of wireless communications and his research lab IRIDA, where I met some truly brilliant people. Professor Krikidis, for me, is an excellent example of a strategic leader, a leader with exceptional knowledge and tremendous imagination, who leads in a truly distinctive way. Strong and decisive in his leadership, yet never imposing or overbearing. He constantly pushes everyone to improve, explains life through science, and motivates people to develop their ideas and turn them into real, impactful outcomes. From the very first time we discussed my thesis topic, I became fascinated by it, imagining myself diving deep into an unknown field for me, known as quantum computing feeling like a real life scientist.

Continuing, I would like to especially thank my friend Panagiotis, who helped me set up my working environment and supported me with the fundamentals of machine learning and programming interfaces. Next, I want to thank Triantafyllos for his significant contribution to my thesis, working together on the code and helping me with the autoencoders, which are part of his private research. I would also like to thank Anna Maria for her contribution to the writing and structuring of this thesis. Lastly, I want to thank my closest friends and family for shaping my personality, being there for me, and keeping me on track to become the best version of myself, disciplined and driven to make them proud. A new chapter in my life now begins, and I am ready to take it all for all those I love, for myself and for those who may no longer be in this life but worked tirelessly so that I could be here today.

ABSTRACT

Nowadays, machine learning (ML) has become an increasingly important tool in wireless communication systems, contributing significantly to recent research developments. In prior work, wireless communication systems have been modeled as end-to-end (E2E) autoencoder-based architectures. However, as this direction continues to evolve, the use of large datasets and highly parameterized models has become more prevalent, leading to increased computational complexity and reduced memory efficiency. To address these challenges, quantum machine learning (QML) offers a promising alternative direction, by exploiting principles of quantum physics through quantum computing. The main objective of this work is to examine whether hybrid quantum-classical ML models can reduce the number of trainable parameters in wireless communication systems while maintaining performance comparable to their classical counterparts. To this end, we analyze the performance of the hybrid autoencoder (HAE) in comparison with the classical autoencoder (CAE) across a variety of channel models and architectural modifications in both linear and nonlinear communication channels, while preserving the lower parameter count of the HAE. Our results demonstrate that it is possible to achieve comparable performance while using significantly fewer trainable parameters.

Keywords: Wireless Communications, Autoencoders, Hybrid Quantum Classical Machine Learning

TABLE OF CONTENTS

ABSTRACT	iii
TABLE OF CONTENTS	iv
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	x
1 Introduction	1
1.1 Fundamentals of Quantum Computing	1
1.2 Literature Review	2
1.3 Research Questions and Contributions	3
1.4 Outline	4
2 Background	5
2.1 Classical Digital Communications	5
2.1.1 Transmitter	5
2.1.2 Constellation Shaping	8
2.1.3 Digital-to-Analog Conversion (DAC)	8
2.1.4 RF Up-Conversion	8
2.2 Communication Channels	9
2.2.1 Rayleigh Fading	9
2.2.2 Rician Fading	10
2.2.3 3GPP Fading	10
2.2.4 Additive White Gaussian Noise (AWGN)	11
2.2.5 Receiver	11
2.3 Performance Metrics	12
2.3.1 Signal-to-Noise Ratio (SNR)	12
2.3.2 Symbol Error Rate (SER)	12

2.3.3	Block Error Rate (BLER)	13
2.4	Classical Machine Learning and Deep Learning	13
2.4.1	Types of Deep Learning	13
2.5	Neural Networks (NNs)	15
2.5.1	Feedforward NNs (FNNs)	15
2.5.2	Stochastic Gradient Descent (SGD)	15
2.5.3	Layers	16
2.5.4	Activation Functions	16
2.5.5	Loss Functions	17
2.6	Autoencoders for End-to-End Communication Systems	18
2.6.1	Message Representation	19
2.6.2	Encoder (Transmitter)	19
2.6.3	Channel	20
2.6.4	Decoder (Receiver)	20
2.7	Fundamental of Quantum Computing	20
2.7.1	Qubit vs Classical Bit	21
2.7.2	Qubits and Quantum State Representation	21
2.7.3	Dirac Notation (Bra–Ket Formalism)	21
2.7.4	Superposition	22
2.7.5	Hilbert Space	22
2.7.6	Bloch Sphere Representation	22
2.7.7	Multiple Qubits and Two-Qubit Superposition	23
2.7.8	Quantum Entanglement	24
2.7.9	Quantum Gates	25
2.8	From Classical Machine Learning to Quantum Machine Learning	27
2.9	QML Categories	27
2.10	Deep Quantum Neural Networks (DQNN)	28
2.11	Parameterized Quantum Circuits (PQCs)	29
2.12	Training a QNN / Computational Enviroment	30
2.12.1	High-Performance Computing (HPC)	30
2.12.2	SLURM Workload Manager	30
2.12.3	CPU vs GPU Architecture	31
3	Framework Analysis and System Models	33
3.1	CAE vs HAE over Fading Channels	33
3.1.1	Common Framework for the Autoencoders	33
3.1.2	Classical Encoder Architecture	34

3.1.3	Quantum Encoder Architecture	35
3.2	An AE-based Framework for Non-linear UR-SWIPT Channels	39
3.2.1	System Model	41
3.2.2	Algorithmic Reference Model	42
4	Modified Quantum Encoder Designs	44
4.1	Lower-Parameter Quantum Encoder Variants	44
4.2	Building a new Quantum Encoder from Scratch	46
4.2.1	Decoder parameter reduction	46
4.2.2	Custom PQC Ansatz Design	47
4.2.3	Implemented Encoder Design	49
4.2.4	Parameter Budget Control	51
4.3	Hybrid UR-SWIPT	51
5	Numerical Results and Discussion	53
5.1	Linear Channel (HAE vs CAE)	53
5.1.1	Parameters	53
5.1.2	Parameter savings	54
5.1.3	BLER vs SNR Performance	54
5.2	Lower-Parameter Quantum Encoder Variants	56
5.3	Results of the new Quantum Encoder	57
5.3.1	BLER vs Epoch	57
5.3.2	BLER over the Rayleigh channel	59
5.4	Non-Linear Channel (UR-SWIPT)	59
5.4.1	Parameter saving	59
5.4.2	Performance Evaluation	60
5.5	Conclusion	61
A	Appendix	64
A.1	HPC Workflow and Experimental Infrastructure	64
A.2	Simulator Backend Choice for HPC Execution	64
A.3	Creating a Quantum Device	65
A.4	Additional Results	66
	BIBLIOGRAPHY	69

LIST OF TABLES

2.1	Training algorithm of a Deep Quantum NN (DQNN).	32
3.1	Step-by-step amplitude embedding example for $k = 2$, $M = 4$, and $n = 3$ qubits.	36
3.2	Example of amplitude embedding for $k = 2$ and $n = 3$ qubits.	37
5.1	Comparison of parameters size between CAE and HAE architectures for different (n, k) configurations.	54
5.2	Parameter comparison for the $(7, 4)$ original quantum encode and its low-parameter PQC variants.	56
5.3	Parameter comparison between CAE and HAE for the UR-SWIPT ID model in the $M = 8$, $P_t = -2.5$ dBm setup.	60
A.1	Trainable parameter comparison between the CAE and HAE models in the UR-SWIPT setting for $M = 4$	66

LIST OF FIGURES

2.1	Structure of a digital communication system.	5
2.2	Block diagram of a conventional transmitter. Blocks highlighted in green operate in the digital domain, while the rest (i.e., blocks in red) operate in the analog domain.	5
2.3	Block diagram of a classical wireless receiver. Blocks highlighted in red operate in the analog domain, blocks highlighted in green operate in the digital domain, and the ADC marks the transition between the two domains.	11
2.4	Classification example: a NN assigns an input object to a shape category. . . .	14
2.5	Regression example: continuous prediction of received signal power as a function of transmission distance.	14
2.6	End-to-end autoencoder architecture for communication systems.	19
2.7	CNOT gate illustrating the control and target qubits.	26
2.8	Categories of QML based on types of data and processor.	28
2.9	Deep Quantum NN (DQNN) architecture.	29
2.10	Hybrid classical–quantum (CQ) parameterized quantum circuit architecture. . .	30
3.1	CAE architecture for E2E communication system.	34
3.2	Hybrid quantum–classical autoencoder architecture with unified channel block.	35
3.3	Structure of a single PQC layer with parameterized rotations, nearest-neighbor entanglement, and Pauli- Z measurements.	38
3.4	UR-SWIPT circuit.	40
3.5	UR-SWIPT architecture.	42
4.1	Schematic of the P12 low-parameter PQC variant. Within each layer, all even-indexed qubits share one trainable rotation parameter and all odd-indexed qubits share another.	45
4.2	Schematic of the P18 low-parameter PQC variant. The qubits are divided into three fixed groups, providing more flexibility than P12 while still keeping the encoder compact.	46

4.3	Structure of the implemented custom quantum encoder. The binary representation of the input symbol is used both for angle embedding and for the generation of control signals that adapt the variational layers and the readout stage of each branch.	50
4.4	Hybrid quantum encoder architecture used in the UR-SWIPT framework. . . .	52
5.1	Rayleigh-channel comparison among HAE, CAE, and the corresponding baseline references for the (4, 4), (7, 4), and (8, 8) configurations. Here, the baseline refers to BPSK for the (4, 4) and (8, 8) settings, and to the Hamming code for the (7, 4) setting.	54
5.2	Rician-channel comparison among HAE, CAE, and the corresponding baseline references for the (4, 4), (7, 4), and (8, 8) configurations. Here, the baseline refers to BPSK for the (4, 4) and (8, 8) settings, and to the Hamming code for the (7, 4) setting.	55
5.3	3GPP-channel comparison among HAE, CAE, and the corresponding baseline references for the (4, 4), (7, 4), and (8, 8) configurations. Here, the baseline refers to BPSK for the (4, 4) and (8, 8) settings, and to the Hamming code for the (7, 4) setting.	55
5.4	BLER versus SNR comparison between the original HAE(7,4) model and the low-parameter P12 variant over the Rayleigh channel.	57
5.5	BLER versus SNR comparison between the original HAE(7,4) model and the low-parameter P18 variant over the Rayleigh channel.	58
5.6	BLER versus SNR comparison between the original HAE(7,4) model and the low-parameter P18 variant across the full evaluation setting.	58
5.7	Training behavior of the custom quantum encoder, showing BLER as a function of the training epoch.	59
5.8	BLER as a function of the training epoch for the reference hybrid model reported by Zhang et al.	60
5.9	BLER versus SNR performance of the custom quantum encoder.	61
5.10	SER versus SNR comparison for the UR-SWIPT ID task, including the algorithmic reference, the learned classical model, and the HAE implementation. . .	62
A.1	HPC execution workflow using SLURM on CPU-based compute nodes for CAE and HAE experiments, including post-processing and visualization.	65
A.2	SER performance of the Hybrid UR-SWIPT model for the $M = 4$ setting. . . .	67
A.3	Learned constellation of the Hybrid UR-SWIPT model for the $M = 4$ setting. .	68

LIST OF ABBREVIATIONS

ML	Machine Learning
NN	Neural Networks
ADC	Analog-to-Digital Converter
AE	Autoencoder
AWGN	Additive White Gaussian Noise
BLER	Block Error Rate
BPSK	Binary Phase Shift Keying
CAE	Classical Autoencoder
HAE	Hybrid Autoencoder
CC	Classical–Classical
CQ	Classical–Quantum
QC	Quantum–Classical
QQ	Quantum–Quantum
DAC	Digital-to-Analog Converter
DQNN	Deep Quantum Neural Network
DSP	Digital Signal Processing
E2E	End-to-End
ENOB	Effective Number of Bits
FNN	Feedforward Neural Network
GCS	Geometric Constellation Shaping
GPU	Graphics Processing Unit
HPC	High-Performance Computing
LoS	Line-of-Sight
ML	Machine Learning
MSE	Mean Squared Error
NISQ	Noisy Intermediate-Scale Quantum
PCS	Probabilistic Constellation Shaping

PQC Parameterized Quantum Circuit
QC Quantum Computing
QML Quantum Machine Learning
QNN Quantum Neural Network
RF Radio Frequency
SER Symbol Error Rate
SGD Stochastic Gradient Descent
SLURM Simple Linux Utility for Resource Management
SNDR Signal-to-Noise-and-Distortion Ratio
SNR Signal-to-Noise Ratio
FFNs Feedforward neural networks
DNN Deep Neural Network
CP Completely Positive
URSWIPT Unified Receiver Simultaneous Wireless Information and Power Transfer
ID Information Decoding
EH Energy Harvesting
QML Quantum Machine Learning
DL Deep Learning
QNNs Quantum Neural Networks

1 Introduction

Wireless communication is the transmission of information without the need for any physical cable or wire connection. It is one of the key technologies currently utilized in our daily lives because it supports various applications, including WiFi connections, mobile connections, and even satellite communications. The main parts of a wireless communication system have been optimized based on their unique requirements to fulfill their duties. For example, mobile wireless communication requires efficient cryptography, robust user authentication mechanisms, and noise filtering.

There are many constraints on wireless communication that may harm the quality of signals. These limitations could also affect the performance and security of the wireless systems. However, the rapid development of deep learning techniques has allowed the optimization of the classic wireless communication systems. Recently, researchers have modeled the complete wireless communication system as an E2E autoencoder, where the transmitter and receiver are implemented using neural networks.

These DL-based approaches have shown promising results compared to conventional communication methods [1, 2]. In particular, they offer the potential to improve system performance by jointly optimizing the transmitter and receiver, rather than designing each component separately. This E2E learning framework allows the system to adapt more effectively to channel conditions and impairments, potentially leading to enhanced robustness and efficiency.

However, as these systems evolve, the increasing number of layers they consist of leads to a significant growth in the number of parameters, resulting in high computational complexity [3]. This is where the properties of quantum physics are leveraged through quantum computing. By transforming classical neural networks (NNs) into quantum neural networks (QNNs), that is, transitioning from classical machine learning to quantum machine learning (QML), it becomes possible to explore whether the number of parameters can be reduced while maintaining comparable performance.

1.1 Fundamentals of Quantum Computing

The concept of quantum computation was first proposed by Richard Feynman, who suggested that quantum systems could be efficiently simulated only by other quantum systems of comparable size [4]. This idea laid the foundation for quantum computing as a new computational paradigm.

Over the years, several quantum algorithms have been proposed with the aim of solving spe-

cific computational tasks more efficiently than classical supercomputers. Notable breakthroughs include the Deutsch–Jozsa algorithm [5], Shor’s factoring algorithm [6], and Grover’s search algorithm [7]. These algorithms demonstrated, at least theoretically, that quantum computers can achieve exponential or quadratic speedups for certain classes of problems.

Initially, most developments in quantum computing remained largely theoretical. However, in recent years, experimental progress has led to the development of Noisy Intermediate-Scale Quantum (NISQ) devices, enabling practical quantum information processing under realistic hardware constraints [8]. These devices typically operate with on the order of 50–100 qubits and are characterized by limited coherence times and gate fidelities.

Although current quantum processors can contain around one hundred qubits, many such systems can still be simulated classically under certain conditions. In 2019, the 53-qubit *Sycamore* processor developed by Google Quantum AI performed a random circuit sampling task that was claimed to be infeasible for classical supercomputers at the time. This experiment was widely regarded as a milestone toward demonstrating quantum computational advantage.

As noted by Preskill [8],

“The 100-qubit quantum computer will not change the world right away — we should regard it as a significant step toward the more powerful quantum technologies of the future.”

1.2 Literature Review

A foundational step toward learning-based wireless communication systems was introduced by O’Shea and Hoydis in [1]. Their work proposed a new way to model the physical layer by treating the transmitter, channel, and receiver as a single end-to-end system trained jointly as an autoencoder. This approach showed that communication schemes can be learned directly from data, without explicitly designing separate modulation and decoding blocks. However, the paper also highlighted an important limitation: as the system scales to larger message sets and more complex settings, the training complexity and the number of trainable parameters increase significantly, making the approach harder to apply in larger practical scenarios.

A direction for addressing this limitation was later introduced by Zhang et al. in [3]. In their work, the classical encoder of the end-to-end communication system is replaced by a quantum encoder, while the decoder remains classical, resulting in a hybrid quantum-classical autoencoder. The main motivation is to reduce the number of trainable parameters while preserving the communication performance of the original classical model. Their results over linear wireless channels, including Rayleigh, Rician, and 3GPP fading settings, show that the hybrid architecture can achieve performance close to that of the classical autoencoder while using

fewer parameters. This makes the work particularly important, since it demonstrates that hybrid quantum-classical learning can offer a more compact alternative to purely classical implementations in linear communication systems.

The practical implementation of such hybrid models relies on suitable quantum software and computational infrastructure. Asadi et al. in [9] present the PennyLane Lightning framework, a high-performance simulation suite for parametrized quantum circuits. Their work shows that optimized simulators such as `lightning.qubit` are especially useful for repeated circuit evaluations and gradient-based hybrid quantum-classical training. In a similar fashion, as highlighted in [10], assessing the performance of CPUs and GPUs solely in terms of computation is inadequate. The reason for this lies in the fact that data transfer is an essential aspect of running a GPU program. Specifically, it has been demonstrated by the authors that the cost of moving data from the CPU to the GPU may actually reduce or overshadow the advantages offered by GPU acceleration. Therefore, a proper assessment of CPU and GPU implementations should account for both kernel runtime and memory transfer costs.

When moving beyond the baseline hybrid architecture and designing new quantum encoders, the measurement stage also becomes an important design consideration. Chen et al. in [11] argue that quantum measurement should not always be treated as a fixed and passive readout step. Instead, they show that the measurement mechanism can be programmable and can serve as an active part of the model design. Their work highlights that the final performance of a quantum learning model depends not only on the embedding and variational circuit, but also on the way the final quantum state is measured. This perspective is particularly useful when constructing custom quantum encoders, where the measurement stage may be considered as part of the encoder architecture itself.

Finally, the extension of autoencoder-based communication systems to nonlinear wireless scenarios is addressed by Mavroultos et al. in [12]. Their work studies Unified Receiver Simultaneous Wireless Information and Power Transfer (UR-SWIPT), a nonlinear communication setting in which the transmitted signal must support both information decoding (ID) and energy harvesting (EH). The system is modeled as an E2E AE and is trained to learn signal constellations that balance these two objectives. The results show that autoencoder-based learning can remain effective even in nonlinear channels, providing a useful framework for studying both communication reliability and harvested energy in a unified way.

1.3 Research Questions and Contributions

In order to accomplish this objective, this thesis answers some research questions and contributes accordingly. Specifically, this work first seeks to confirm the validity of the key observations

made regarding hybrid quantum-classical learning models applied in wireless communications under linear channels. Next, it analyzes the possibility of minimizing the trainable parameters in the quantum encoder without sacrificing its performance drastically. Moreover, this thesis aims to explore whether it is feasible to minimize the number of parameters in the classical decoder and simultaneously enhance the quantum encoder such that the overall number of parameters in the hybrid model remains constant or decreases while maintaining its comparable performance. Lastly, this thesis assesses the efficiency of hybrid learning models in non-linear wireless communications, particularly the decoding of information in the UR-SWIPT setting.

1.4 Outline

The remainder of this thesis is organized as follows. Chapter 2 presents the background material required for understanding the problem, including the necessary concepts from wireless communications, machine learning, and quantum computing. Chapter 3 describes the system models and the main architectures examined in this work. Chapter 4 presents the methodology, including the simulation setup, implementation details, and evaluation procedure. Chapter 5 discusses the experimental results for both linear and nonlinear channel settings, with emphasis on parameter efficiency and performance comparison. Finally, Chapter 6 concludes the thesis and outlines possible directions for future work.

2 Background

2.1 Classical Digital Communications

The primary objective of a communication system is to reliably transmit information from one point to another, where the information is typically represented as a sequence of bits. To achieve this goal, the transmitter converts the digital message (bit sequence) into an analog waveform suitable for transmission over a physical channel. This process ensures that the transmitted signal is robust against channel impairments through appropriate channel coding and modulation schemes. At the receiver's side, the analog signal is processed and decoded to reconstruct the original digital message as accurately as possible. In the following subsections, we provide a detailed description of each component of the system. Figure 2.1 illustrates the structure of a digital communication system, which consists of three main blocks: the transmitter, the channel, and the receiver.



Figure 2.1: Structure of a digital communication system.

2.1.1 Transmitter

We begin with the transmitter side. Fig. 2.2 shows the block diagram of a conventional transmitter in a digital communication system. The transmitter takes the information bits and converts them into a waveform that can be sent through the physical channel. The first processing stages, highlighted in green, operate in the digital domain and include channel coding, modulation, constellation mapping, and baseband processing. These stages prepare the digital symbols that carry the transmitted information. The signal is then converted into analog form by the digital-to-analog converter (DAC). After that, the RF front-end performs up-conversion and the required analog processing to produce the final transmitted waveform. The main transmitter blocks are described in more detail in the following subsections.

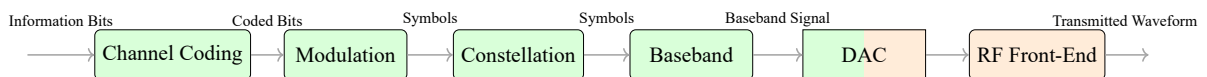


Figure 2.2: Block diagram of a conventional transmitter. Blocks highlighted in green operate in the digital domain, while the rest (i.e., blocks in red) operate in the analog domain.

2.1.1.1 Channel Coding

Channel coding is used in digital communication systems to improve transmission reliability by adding controlled redundancy to the transmitted data. Instead of sending the original information bits directly, the encoder maps blocks of k bits into longer codewords of length n , where $n > k$ [13]. As a result, the transmitted sequence is not the raw message itself, but a structured codeword that helps the receiver detect and correct errors.

The ratio

$$R = \frac{k}{n} \quad (2.1)$$

is defined as the *code rate*, quantifying the trade-off between redundancy and spectral efficiency. Lower code rates correspond to higher redundancy and increased robustness against channel impairments.

The performance of a linear block code is determined by its minimum Hamming distance

$$d_{\min} = \min_{\mathbf{c}_i \neq \mathbf{c}_j} d(\mathbf{c}_i, \mathbf{c}_j), \quad (2.2)$$

which corresponds to the minimum number of differing bit positions between any two distinct codewords. For linear codes, this reduces to the minimum Hamming weight among all non-zero codewords, significantly simplifying performance analysis.

The minimum distance directly determines the error-correction capability of the code. Specifically, a linear block code can correct up to

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor \quad (2.3)$$

bit errors per codeword.

Hamming codes constitute a specific class of linear block error-correcting codes designed to detect and correct errors introduced during transmission over noisy channels. A binary Hamming code is typically denoted as an (n, k) block code, where

$$n = 2^m - 1, \quad k = n - m, \quad (2.4)$$

with parity bits added according to a predefined linear structure. Hamming codes are characterized by a minimum Hamming distance $d_{\min} = 3$, enabling single-error correction.

By incorporating reliability information, soft-decision decoding significantly improves the error-correction performance of Hamming codes compared to hard-decision decoding, particularly

in low and moderate signal-to-noise ratio (SNR) regimes. As a result, Hamming codes with soft-decision decoding serve as a classical and well-established benchmark for evaluating the performance of more advanced coding and learning-based communication schemes.

2.1.1.2 Modulation

Modulation is the process by which digital information is mapped onto physical waveforms suitable for transmission over a communication channel. After channel coding, groups of m bits are assigned to complex-valued symbols selected from a predefined set known as a *constellation*. Each transmitted symbol can be expressed as

$$x = x_I + jx_Q, \quad (2.5)$$

where x_I and x_Q denote the in-phase and quadrature components, respectively.

The modulation format determines both the number of constellation points M and the number of bits per symbol

$$m = \log_2 M, \quad (2.6)$$

thereby directly influencing the achievable data rate and spectral efficiency of the system.

Higher-order modulation schemes increase throughput by transmitting more bits per symbol. However, as M increases, the Euclidean distance between constellation points (for fixed transmit power) typically decreases, rendering the system more sensitive to noise and channel impairments. Consequently, the choice of modulation represents a fundamental trade-off between data rate, power efficiency, and reliability in digital communication systems.

- **Binary Phase Shift Keying (BPSK):** Binary Phase Shift Keying (BPSK) is one of the simplest and most widely used digital modulation schemes. Whereas, each transmitted bit is mapped onto one of two possible carrier phases, separated by π radians. As a result, the transmitted signal assumes one of two antipodal constellation points.

At the baseband level, BPSK modulation can be represented by the symbol alphabet

$$x \in \left\{ +\sqrt{E_b}, -\sqrt{E_b} \right\}, \quad (2.7)$$

where E_b is the energy per transmitted bit. A binary value of “1” and “0” is typically mapped to $+\sqrt{E_b}$ and $-\sqrt{E_b}$, respectively.

2.1.2 Constellation Shaping

Constellation shaping originates from information theory and the foundational work of Shannon on channel capacity [14]. Since all practical communication channels introduce distortions and noise, the maximum achievable data rate is fundamentally limited by the channel capacity, which quantifies the maximum amount of information that can be reliably transmitted over a given channel.

Constellation shaping aims to reduce the so-called *shaping gap*, i.e., the performance loss between practical discrete constellations and the capacity-achieving Gaussian input distribution. This is achieved by optimizing either the geometry or the probability distribution of the constellation points.

Broadly, shaping techniques are categorized into two main classes:

- **Geometric Constellation Shaping (GCS):** The positions of constellation points are modified while maintaining equiprobable signaling.
- **Probabilistic Constellation Shaping (PCS):** Constellation points lie on a fixed geometric grid, but are transmitted with non-uniform probabilities.

2.1.3 Digital-to-Analog Conversion (DAC)

Digital baseband signals cannot be directly radiated over a wireless channel; therefore, a Digital-to-Analog Converter (DAC) is required to transform discrete-time digital samples into continuous-time analog waveforms prior to transmission. The performance of a DAC is commonly characterized by parameters such as resolution (number of quantization bits), sampling frequency, Signal-to-Noise-and-Distortion Ratio (SNDR), and Effective Number of Bits (ENOB).

2.1.4 RF Up-Conversion

After DAC conversion, the continuous-time baseband signal is translated to the carrier frequency through an up-conversion stage. In complex notation, the modulated RF signal can be expressed as

$$s(t) = \text{Re} \{ x(t) e^{j2\pi f_c t} \}, \quad (2.8)$$

where $x(t)$ denotes the complex baseband waveform and f_c is the carrier frequency.

In practical wireless systems, this operation is implemented using mixers, local oscillators, and power amplifiers, forming the RF front-end responsible for frequency translation and signal amplification prior to radiation.

2.2 Communication Channels

The communication channel is the physical medium through which the transmitted signal travels from the transmitter to the receiver. During propagation, the signal is affected by distortions and random variations that influence its amplitude, phase, and overall reliability. These effects may be caused by thermal noise, multipath propagation, Doppler shifts, attenuation, shadowing, phase noise, or other physical phenomena, depending on the transmission environment [13].

To study system performance and design reliable transmission methods, the channel is usually described through mathematical models. These models range from simple additive noise models to more complex fading models that capture time-varying channel behavior. The choice of model depends on factors such as the propagation environment, carrier frequency, user mobility, and system requirements.

In the following, we review the most common channel models used in wireless communications [15, 16], focusing on those that are directly relevant to the scenarios studied in this thesis.

2.2.1 Rayleigh Fading

Rayleigh fading is a widely used statistical model for characterizing the amplitude variations of the received signal in propagation environments where no direct line-of-sight (LoS) path exists between the transmitter and the receiver. Such conditions typically arise in urban or indoor scenarios, where the received signal is formed by the superposition of a large number of scattered and reflected multipath components.

Under rich scattering conditions, the in-phase and quadrature components of the complex channel gain can be modeled as independent zero-mean Gaussian random variables with equal variance σ^2 , i.e.,

$$X_1, X_2 \sim \mathcal{N}(0, \sigma^2). \quad (2.9)$$

The envelope of the received signal is therefore

$$R = \sqrt{X_1^2 + X_2^2}, \quad (2.10)$$

which follows a Rayleigh distribution. Equivalently, the complex channel coefficient can be written as

$$h \sim \mathcal{CN}(0, 2\sigma^2). \quad (2.11)$$

The received signal is:

$$y = hx + n \quad (2.12)$$

where x is the transmitted signal, h is the complex channel gain modeling fading and n is additive white Gaussian noise.

2.2.2 Rician Fading

The Rician distribution is suitable for modeling fluctuations in amplitude when there is a dominant LoS path along with scattered components between the transmitter and receiver. The complex baseband received signal is given by:

$$r = se^{j\theta} + x, \quad (2.13)$$

where $se^{j\theta}$ is the deterministic LoS component with amplitude s , and $x \sim \mathcal{CN}(0, 2\sigma^2)$ represents the scattered components.

By writing $x = X_1 + jX_2$, where $X_1, X_2 \sim \mathcal{N}(0, \sigma^2)$, the received signal becomes:

$$r = (s + X_1) + jX_2. \quad (2.14)$$

The envelope of the received signal is:

$$R = \sqrt{(X_1 + s)^2 + X_2^2}, \quad (2.15)$$

where $X_1, X_2 \sim \mathcal{N}(0, \sigma^2)$.

The Rician K -factor, defined as the ratio of the power of the LoS component to the scattered power, is:

$$K = \frac{s^2}{2\sigma^2}. \quad (2.16)$$

2.2.3 3GPP Fading

The channel models adopted in 3GPP cellular standards are based on classical stochastic fading models that describe the statistical behavior of wireless propagation in multipath environments. These models build upon the fundamental concepts of additive noise and small-scale fading.

In particular, the 3GPP framework employs:

- Rayleigh fading for modeling non-line-of-sight (NLoS) propagation scenarios, where the received signal consists of multiple scattered components with no dominant direct path.

- Rician fading for scenarios where a strong LoS component is present in addition to scattered multipath contributions.

2.2.4 Additive White Gaussian Noise (AWGN)

In the additive white Gaussian noise (AWGN) channel model, the received signal $r(t)$ is expressed as

$$r(t) = s(t) + n(t), \quad (2.17)$$

where $s(t)$ is the transmitted signal and $n(t)$ is an additive noise component.

The noise process $n(t)$ is modeled as white and Gaussian. Specifically, it is assumed to be a zero-mean Gaussian random process with variance

$$\sigma^2 = \frac{N_0}{2} \quad (2.18)$$

per real dimension.

The term *white* indicates that the noise has a flat power spectral density over the frequency band of interest, while *Gaussian* refers to its normal probability distribution.

2.2.5 Receiver

Fig. 2.3 illustrates the fundamental blocks of a classical wireless receiver comprising both analog and digital processing stages. The received RF signal is first processed by the analog front-end, where amplification, frequency down-conversion, and filtering are performed in order to obtain a baseband analog signal.

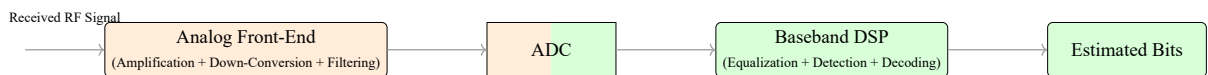


Figure 2.3: Block diagram of a classical wireless receiver. Blocks highlighted in red operate in the analog domain, blocks highlighted in green operate in the digital domain, and the ADC marks the transition between the two domains.

The baseband signal is then digitized using an Analog-to-Digital Converter (ADC), marking the transition from the analog to the digital domain.

Assuming a narrowband flat-fading channel, the discrete-time baseband received signal can be expressed as

$$y[k] = h[k]x[k] + n[k], \quad (2.19)$$

where $h[k]$ denotes the complex channel coefficient modeling small-scale fading and

$$n[k] \sim \mathcal{CN}(0, N_0) \quad (2.20)$$

represents the AWGN. After digitization, digital signal processing (DSP) techniques are applied to mitigate channel impairments. These typically include channel estimation and equalization, symbol detection, and channel decoding, ultimately enabling the recovery of the transmitted information bits.

2.3 Performance Metrics

2.3.1 Signal-to-Noise Ratio (SNR)

The Signal-to-Noise Ratio (SNR) is defined as the ratio between the average signal power and the average noise power at the receiver. In linear scale, it is expressed as

$$\text{SNR} = \frac{P_s}{P_n}, \quad (2.21)$$

where P_s is the average signal power and P_n the average noise power.

For an AWGN channel, SNR is commonly written in terms of symbol energy and noise spectral density as

$$\text{SNR} = \frac{E_s}{N_0}. \quad (2.22)$$

In decibel (dB) scale, it is given by

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{E_s}{N_0} \right). \quad (2.23)$$

2.3.2 Symbol Error Rate (SER)

The Symbol Error Rate (SER) is defined as the probability that a transmitted symbol is incorrectly detected at the receiver:

$$\text{SER} = \Pr(\hat{s} \neq s), \quad (2.24)$$

where s is the transmitted symbol and $\hat{s}$ its detected estimate. SER constitutes the primary performance metric for M -ary modulation schemes in classical digital communication systems.

2.3.3 Block Error Rate (BLER)

The Block Error Rate (BLER) is defined as the probability that at least one symbol within a transmitted block is detected incorrectly:

$$\text{BLER} = \Pr(\hat{\mathbf{s}} \neq \mathbf{s}), \quad (2.25)$$

where $\mathbf{s}$ denotes the transmitted symbol block and $\hat{\mathbf{s}}$ its decoded estimate. Equivalently, BLER can be expressed as

$$\text{BLER} = 1 - \Pr(\text{all symbols in the block are correctly detected}). \quad (2.26)$$

2.4 Classical Machine Learning and Deep Learning

2.4.1 Types of Deep Learning

2.4.1.1 Supervised Learning

Supervised learning is a type of learning in which a model is trained using labeled data. This means that each input sample is given together with its correct output. The goal is to learn the relationship between the inputs and the targets, so that the model can make reliable predictions for new data it has not seen before [17].

Depending on the nature of the output variable, supervised learning problems are commonly divided into two principal categories: classification and regression [18].

Figures 2.4 and 2.5 illustrate two representative machine learning tasks: classification (top) and regression (bottom). In the classification example (Fig. 2.4), the goal is to assign an input sample to one of a finite number of predefined categories. Consider a simple dataset consisting of geometric shapes, where each sample belongs to a specific class. The classification model processes the input and outputs a discrete label indicating the predicted category. In this example, the neural network receives a shape as input and determines whether it belongs to the triangle or square class. Since the output is restricted to a limited set of possible categories, the problem is considered categorical.

In contrast, regression aims to predict a continuous output value. For example, consider the problem of estimating the received signal power based on the transmission distance. The regression model learns a functional relationship between the input variable x (distance) and the output variable y (received power). Unlike classification, the output is not restricted to predefined categories but can take any value within a continuous range.

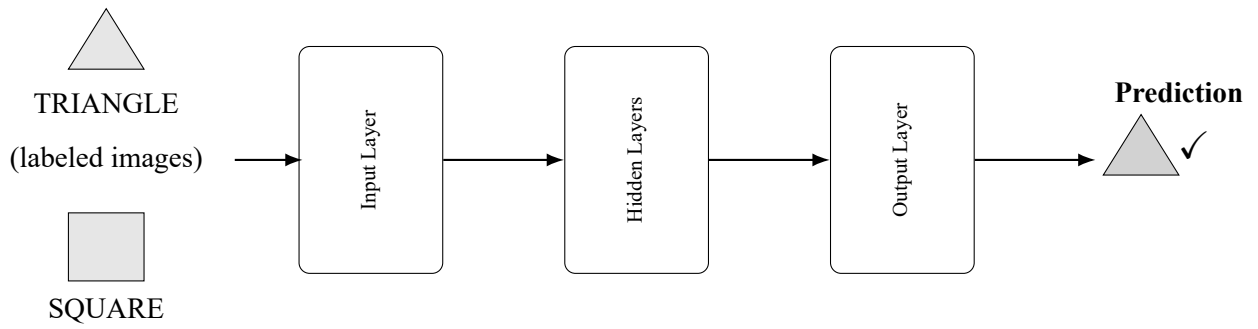


Figure 2.4: Classification example: a NN assigns an input object to a shape category.

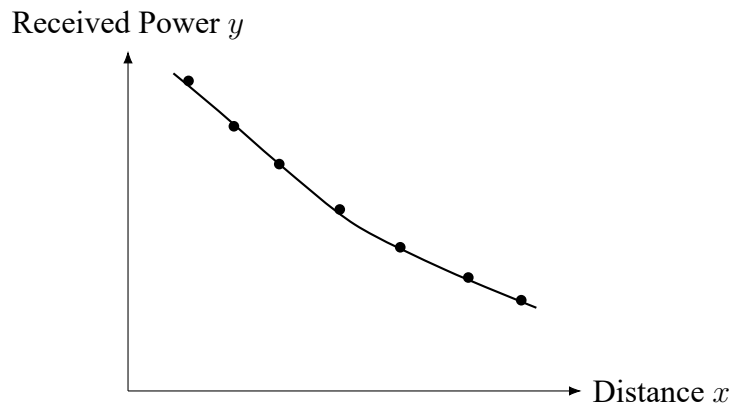


Figure 2.5: Regression example: continuous prediction of received signal power as a function of transmission distance.

2.4.1.2 Unsupervised Learning

Unlike supervised learning algorithms, which depend on labeled datasets during training, unsupervised learning methods operate solely on unlabeled data. Their primary objective is to uncover latent structures, hidden patterns, or intrinsic relationships within the dataset without relying on predefined target labels.

A fundamental application of unsupervised learning is clustering, whose objective is to detect natural groupings within an unlabeled dataset. Consider, for example, a dataset containing images of cars and bikes, where class labels are not available. A clustering algorithm attempts to partition the data into two distinct groups by identifying shared features among similar samples. However, since no labeled information is provided during training, the algorithm cannot determine which cluster corresponds to cars or bikes semantic interpretation must be performed afterward.

2.5 Neural Networks (NNs)

Neural networks (NNs) are models that can learn complex nonlinear relationships by combining many simple operations. Because of this flexibility, they have become one of the most important tools in modern ML. In this section we introduce the basic concepts needed to understand NN-based communication systems. We first present the structure of feedforward NNs (FNNs) and their mathematical formulation. We then describe the training procedure based on Stochastic Gradient Descent (SGD), followed by a discussion of fundamental building blocks such as layers, activation functions, and loss functions. Finally, we introduce classical autoencoders, which form the foundation for E2E learning approaches in communication systems.

2.5.1 Feedforward NNs (FNNs)

A FNN represents the simplest NN architecture. It maps an input vector to an output vector through a sequence of layers. In this setup, the input is processed sequentially, where the output of one layer serves as the input to the next, until the final output is produced. Information flows strictly in one direction, from the input layer to the output layer.

Each layer performs a linear transformation followed by a activation function. Mathematically, the operation of layer l can be expressed as

$$\mathbf{r}_l = \sigma(\mathbf{W}_l \mathbf{r}_{l-1} + \mathbf{b}_l), \quad (2.27)$$

where $\mathbf{r}_{l-1}$ denotes the input to the layer, $\mathbf{W}_l$ and $\mathbf{b}_l$ represent the trainable weight matrix and bias vector, respectively, and $\sigma(\cdot)$ is the activation function.

The parameters $\{\mathbf{W}_l, \mathbf{b}_l\}$ are learned during the training process by minimizing a predefined loss function.

2.5.2 Stochastic Gradient Descent (SGD)

Training a NN involves minimizing a loss function $\mathcal{L}(\boldsymbol{\theta})$ with respect to the model parameters $\boldsymbol{\theta}$. SGD is an iterative optimization algorithm commonly used for this purpose. Starting from an initial parameter vector $\boldsymbol{\theta}_0$, the update rule at iteration t is given by

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla \tilde{\mathcal{L}}(\boldsymbol{\theta}_t), \quad (2.28)$$

where $\eta > 0$ denotes the learning rate, and $\tilde{\mathcal{L}}(\boldsymbol{\theta})$ represents an approximation of the loss function computed over a randomly selected mini-batch of training samples. By selecting a mini-batch size significantly smaller than the total training set, the computational complexity of gra-

dient evaluation is reduced while maintaining efficient convergence. The gradient $\nabla \tilde{\mathcal{L}}(\boldsymbol{\theta}_t)$ is efficiently computed using the backpropagation algorithm, which enables scalable training of DNNs.

2.5.3 Layers

- **Fully Connected (Dense) Layers:** Fully connected (dense) layers apply a linear transformation followed by a activation function:

$$\mathbf{r}_l = \sigma(\mathbf{W}_l \mathbf{r}_{l-1} + \mathbf{b}_l), \quad (2.29)$$

where each neuron in layer l is connected to all neurons in the previous layer. While highly expressive, dense layers require a large number of trainable parameters and may incur significant computational complexity for high-dimensional inputs.

- **Noise Layer:** A noise layer introduces stochasticity by adding Gaussian noise to the input activations without introducing trainable parameters:

$$\mathbf{r}_l = \mathbf{r}_{l-1} + \mathbf{n}, \quad \mathbf{n} \sim \mathcal{N}(\mathbf{0}, \beta \mathbf{I}), \quad (2.30)$$

where β controls the noise variance and $\mathbf{I}$ denotes the identity matrix.

- **Normalization Layer:** A normalization layer rescales the input vector to satisfy a predefined constraint, such as unit energy:

$$\mathbf{r}_l = \frac{\sqrt{N_l} \mathbf{r}_{l-1}}{\|\mathbf{r}_{l-1}\|_2}, \quad (2.31)$$

where N_l denotes the dimensionality of the layer. Such layers do not introduce trainable parameters and are particularly important in communication systems to enforce transmit power constraints.

2.5.4 Activation Functions

- **Linear Activation:** The linear activation function applies an identity mapping to the input:

$$\sigma(u_i) = u_i. \quad (2.32)$$

- **ReLU (Rectified Linear Unit):** The ReLU activation function is defined as

$$\sigma(u_i) = \max(0, u_i). \quad (2.33)$$

It introduces a piecewise linear nonlinearity by preserving positive inputs and suppressing negative values. Due to its simple form and favorable gradient properties for positive activations, ReLU is computationally inexpensive and widely used in DNNs.

- **Softmax Activation:** The softmax activation function converts a vector of real-valued inputs into a probability distribution:

$$\sigma(u_i) = \frac{e^{u_i}}{\sum_j e^{u_j}}. \quad (2.34)$$

It ensures that the outputs are non-negative and sum to one, making it suitable for multi-class classification problems.

- **Sigmoid Activation:** The sigmoid activation function is defined as

$$\sigma(u_i) = \frac{1}{1 + e^{-u_i}}. \quad (2.35)$$

It maps the input into the interval $(0, 1)$ and is commonly used when the output represents probabilities. However, for very large or very small input values, the gradient becomes very small (vanishing gradient problem), which can slow down or hinder the learning process in DNNs.

2.5.5 Loss Functions

To train a NN, a loss function is used to quantify the discrepancy between the network output and the desired target output. The loss function defines the training objective and determines how the network parameters are updated during the learning process.

Given a set of S training samples, the overall loss is defined as the average error over all samples:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{S} \sum_{i=1}^S \ell(\hat{\mathbf{r}}_{L,i}, \mathbf{r}_{L,i}), \quad (2.36)$$

where S denotes the number of training samples, $\hat{\mathbf{r}}_{L,i}$ is the network output at the final layer L for the i -th input, $\mathbf{r}_{L,i}$ is the corresponding target output, and $\ell(\cdot, \cdot)$ is a sample-wise loss function. The parameter set $\boldsymbol{\theta}$ includes all trainable weights and biases of the network. During training, the objective is to minimize $\mathcal{L}(\boldsymbol{\theta})$.

Types of Loss Functions

- **Mean Squared Error (MSE):** The Mean Squared Error (MSE) loss measures the squared Euclidean distance between the target output vector $\mathbf{u}$ and the network output $\mathbf{v}$:

$$\ell(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2^2. \quad (2.37)$$

It is commonly used in regression problems where the network is required to estimate continuous-valued outputs.

- **Categorical Cross-Entropy:** The categorical cross-entropy loss is defined as

$$\ell(\mathbf{u}, \mathbf{v}) = - \sum_j u_j \log(v_j), \quad (2.38)$$

where $\mathbf{u}$ is a one-hot encoded target vector and $\mathbf{v}$ represents the predicted class probabilities (typically obtained via a softmax activation). This loss function is widely used in classification tasks, as it penalizes incorrect probability assignments to the true class and encourages confident predictions.

2.6 Autoencoders for End-to-End Communication Systems

An autoencoder (AE) Fig. 2.6 is an unsupervised learning model composed of two trainable components an encoder and a decoder, which are trained jointly to reconstruct an input through an intermediate representation. In the context of communication systems, the AE is employed to model the entire transmitter–channel–receiver chain as a single E2E system [1].

We consider a finite message set

$$\mathcal{M} = \{0, 1, \dots, M - 1\}, \quad M = 2^k, \quad (2.39)$$

where k is the number of information bits per message. Each message $s \in \mathcal{M}$ is represented by an M -dimensional one-hot vector and is encoded into a signal spanning n channel uses. The corresponding communication rate is the same as the code rate already defined in (2.1).

From an E2E perspective, the communication system can be described as a sequence of three cascaded functions,

$$\hat{s} = f_D(f_h(f_E(s; \theta_E)); \theta_D), \quad (2.40)$$

where f_E is the encoder, f_h is the channel transformation, and f_D the decoder. The trainable parameters of the transmitter and receiver are represented by θ_E and θ_D , respectively.

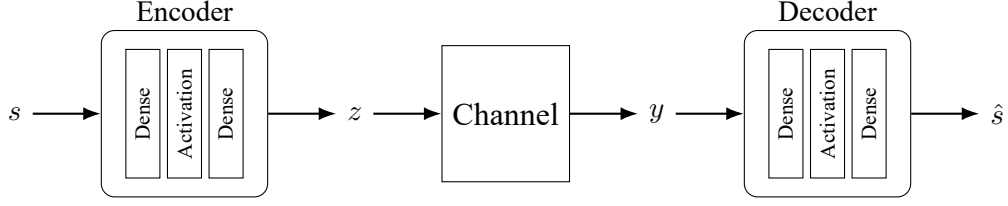


Figure 2.6: End-to-end autoencoder architecture for communication systems.

2.6.1 Message Representation

Before the encoding stage, each source message is represented in one-hot form. More specifically, for a message $s \in \mathcal{M}$, the corresponding input vector $\mathbf{u}_s \in \{0, 1\}^M$ is defined as

$$[\mathbf{u}_s]_i = \begin{cases} 1, & i = s, \\ 0, & i \neq s, \end{cases} \quad (2.41)$$

for $i \in \{0, 1, \dots, M - 1\}$. Therefore, only one entry of the vector is equal to one, while all remaining entries are zero.

2.6.2 Encoder (Transmitter)

The objective of the encoder is to transmit one message $s \in \mathcal{M}$ over a noisy channel using a finite number of channel uses. The encoder can be viewed as a mapping

$$f_E : \mathcal{M} \rightarrow \mathbb{C}^n, \quad (2.42)$$

which maps the discrete message s to a complex-valued transmit vector

$$\mathbf{x} = f_E(s; \theta_E). \quad (2.43)$$

In practice, the encoder receives the one-hot representation of the source message and learns a data-driven mapping to the transmitted signal. The transmitted signal is constrained by the physical limitations of the system, for example through an average energy constraint

$$\|\mathbf{x}\|_2^2 \leq n. \quad (2.44)$$

2.6.3 Channel

The channel models the effect of noise and other impairments on the transmitted signal and is described by the conditional probability density function

$$p(\mathbf{y} \mid \mathbf{x}), \quad (2.45)$$

where $\mathbf{y} \in \mathbb{C}^n$ denotes the received signal. During training, the channel is incorporated as a stochastic layer between the encoder and the decoder. In fading wireless communication systems, the received signal follows the same form introduced earlier in (2.12), adapted here to the vector representation used by the AE.

2.6.4 Decoder (Receiver)

The decoder is implemented as a DNN that maps the received signal vector $\mathbf{y}$ to an estimate of the transmitted message. Its final layer uses a softmax activation function, which produces a probability vector

$$\mathbf{p} = \text{softmax}(\mathbf{z}) \in (0, 1)^M, \quad (2.46)$$

where each entry represents the predicted probability of one possible message. The decoded message $\hat{s}$ is obtained by selecting the message with the highest probability:

$$\hat{s} = \arg \max_{m \in \mathcal{M}} p_m. \quad (2.47)$$

A block error occurs when the decoded message is different from the transmitted one, that is,

$$\hat{s} \neq s. \quad (2.48)$$

Therefore, a good decoder is one that assigns the highest probability to the correct message as often as possible. In this way, minimizing the block error becomes the main performance objective of the receiver.

2.7 Fundamental of Quantum Computing

The term *quantum* refers to a class of physical systems whose behavior cannot be fully explained by classical intuition. At the quantum scale, physical reality is no longer described by deterministic trajectories. Instead, it is characterized by abstract state vectors, probabilistic measurement outcomes, and linear transformations in complex Hilbert spaces. Within this mathematical and physical framework, information acquires a structure that is fundamentally different from that

of classical systems. These principles form the conceptual foundation of quantum computing and quantum information processing [19].

In this section, we begin by discussing the fundamental differences between the quantum bit and the classical bit. We then explain the quantum bit's mathematical representation, and basic quantum principles.

2.7.1 Qubit vs Classical Bit

In a classical system, representing all possible configurations of an n -bit string requires considering 2^n combinations, which must be processed sequentially in conventional computation. A quantum system, however, encodes probability amplitudes for all 2^n basis states simultaneously within a single quantum state. Through quantum interference, amplitudes corresponding to correct solutions can be amplified while others are suppressed.

Quantum computers leverage the principles of superposition and entanglement to represent and process information in ways that differ fundamentally from classical systems. A classical binary bit can assume only one of two discrete values, 0 or 1. In contrast, a qubit can exist in a superposition of both basis states simultaneously.

2.7.2 Qubits and Quantum State Representation

A qubit is described by a state vector in a two-dimensional complex Hilbert space.

This space is spanned by two orthonormal computational basis states:

$$|0\rangle, \quad |1\rangle. \quad (2.49)$$

2.7.3 Dirac Notation (Bra–Ket Formalism)

Quantum states are expressed using Dirac notation. A state vector is written as a “ket” $|\psi\rangle$, representing a column vector in a complex Hilbert space. Its conjugate transpose is called a “bra” and is denoted as $\langle\psi|$.

The inner product between two states is written as

$$\langle\phi|\psi\rangle, \quad (2.50)$$

which yields a complex number corresponding to a probability amplitude.

2.7.4 Superposition

A general pure qubit state can be expressed as a linear combination of the computational basis states:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad (2.51)$$

where $\alpha, \beta \in \mathbb{C}$ are complex probability amplitudes satisfying the normalization condition

$$|\alpha|^2 + |\beta|^2 = 1. \quad (2.52)$$

Upon measurement in the computational basis, the probabilities of obtaining outcomes 0 and 1 are $|\alpha|^2$ and $|\beta|^2$, respectively.

2.7.5 Hilbert Space

A Hilbert space is a complex vector space equipped with an inner product and complete under the induced norm. In quantum mechanics, every physical system is described by a state vector belonging to such a space.

For a single qubit, the relevant Hilbert space is

$$\mathcal{H} = \mathbb{C}^2. \quad (2.53)$$

The inner product encodes physically meaningful quantities such as transition amplitudes and measurement probabilities, while orthogonality between state vectors corresponds to perfectly distinguishable quantum states.

2.7.6 Bloch Sphere Representation

The Bloch sphere provides a geometric representation of pure single-qubit states. Any normalized pure state in $\mathbb{C}^2$ can be written as

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle, \quad (2.54)$$

where $\theta \in [0, \pi]$, and $\phi \in [0, 2\pi)$.

Each point on the surface of the unit sphere corresponds to a normalized state vector. The north and south poles represent the computational basis states $|0\rangle$ and $|1\rangle$, respectively. The polar angle θ determines measurement probabilities, while the azimuthal angle ϕ encodes the relative phase between the basis states. Single-qubit unitary operations correspond to rotations of the state vector on the Bloch sphere.

2.7.7 Multiple Qubits and Two-Qubit Superposition

Just as classical bits become more powerful when arranged in strings, quantum computation becomes significantly more expressive when multiple qubits are considered jointly.

An n -qubit system is described by the tensor product of individual qubit Hilbert spaces. The resulting composite space is

$$\mathcal{H} = (\mathbb{C}^2)^{\otimes n} = \mathbb{C}^{2^n}. \quad (2.55)$$

The computational basis states of such a system are given by

$$|q_1\rangle \otimes |q_2\rangle \otimes \cdots \otimes |q_n\rangle = |q_1 q_2 \cdots q_n\rangle, \quad (2.56)$$

where each $q_i \in \{0, 1\}$ represents the state of the i -th qubit.

For example, in a two-qubit system the computational basis consists of the four orthonormal states

$$|00\rangle, \quad |01\rangle, \quad |10\rangle, \quad |11\rangle. \quad (2.57)$$

2.7.7.1 Two-Qubit Superposition

A general pure two-qubit state can be expressed as a superposition of the computational basis states:

$$|\psi\rangle = \alpha_{00} |00\rangle + \alpha_{01} |01\rangle + \alpha_{10} |10\rangle + \alpha_{11} |11\rangle, \quad (2.58)$$

where $\alpha_{ij} \in \mathbb{C}$ and the normalization condition

$$|\alpha_{00}|^2 + |\alpha_{01}|^2 + |\alpha_{10}|^2 + |\alpha_{11}|^2 = 1 \quad (2.59)$$

ensures that the total probability equals one.

More generally, a pure n -qubit state takes the form

$$|\psi\rangle = \sum_{q_1, \dots, q_n \in \{0, 1\}} \alpha_{q_1 \dots q_n} |q_1 \cdots q_n\rangle, \quad (2.60)$$

where the complex amplitudes $\alpha_{q_1 \dots q_n}$ satisfy

$$\sum_{q_1, \dots, q_n} |\alpha_{q_1 \dots q_n}|^2 = 1. \quad (2.61)$$

2.7.7.2 Measurement Interpretation

In a two-qubit system, the state may exist in a superposition of all four computational basis states simultaneously:

$$|\psi\rangle = \alpha |00\rangle + \beta |01\rangle + \gamma |10\rangle + \eta |11\rangle. \quad (2.62)$$

Each amplitude determines a measurement probability given by the squared magnitude of the coefficient:

$$P(00) = |\alpha|^2, \quad P(01) = |\beta|^2, \quad P(10) = |\gamma|^2, \quad P(11) = |\eta|^2.$$

Upon measurement in the computational basis, the quantum state collapses to one of the basis states with probability determined by the corresponding amplitude.

2.7.8 Quantum Entanglement

Having introduced multi-qubit systems, we are now in a position to define quantum entanglement. Quantum entanglement refers to the nonclassical correlations that can arise between quantum systems. The phenomenon, famously described by Einstein as “spooky action at a distance,” [20] is now recognized as one of the fundamental features of quantum mechanics.

Entanglement occurs when two or more quantum systems are prepared in such a way that their joint state cannot be factorized into a product of individual states [21]. In other words, the state of one subsystem cannot be described independently of the state of the others.

Consider a bipartite quantum system composed of subsystems A and B , with joint Hilbert space

$$\mathcal{H}_{AB} = \mathcal{H}_A \otimes \mathcal{H}_B. \quad (2.63)$$

A pure bipartite state $|\psi\rangle_{AB} \in \mathcal{H}_{AB}$ is said to be *entangled* if it cannot be written in the separable (product) form

$$|\psi\rangle_{AB} = |\psi\rangle_A \otimes |\phi\rangle_B. \quad (2.64)$$

If such a factorization is possible, the state is called *separable* (or product state). Otherwise, it exhibits quantum entanglement.

A paradigmatic example of an entangled state is one of the Bell states:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle), \quad (2.65)$$

which cannot be expressed as a tensor product of two single-qubit states.

Upon measurement, the outcomes of entangled subsystems exhibit correlations that cannot be explained by classical local hidden-variable theories. These correlations violate Bell inequalities and constitute one of the strongest conceptual distinctions between classical and quantum physics.

2.7.9 Quantum Gates

A short discussion of quantum gates is necessary, as they are directly employed in the proposed framework of this work. Understanding their mathematical structure and operational role facilitates the analysis of the parametrized quantum circuit (PQC) architecture presented in a subsequent section.

2.7.9.1 Hadamard Gate (H)

The Hadamard gate is a single-qubit unitary operation that maps the computational basis states to equal superpositions. It is defined by the unitary matrix

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \quad (2.66)$$

Its action on the computational basis states is

$$H |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad H |1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle). \quad (2.67)$$

This gate is fundamental for generating superposition and enabling quantum parallelism.

2.7.9.2 Rotation Gate $R_y(\theta)$

The R_y gate performs a rotation of a qubit state around the y -axis of the Bloch sphere by an angle θ . It is defined as

$$R_y(\theta) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}. \quad (2.68)$$

Equivalently, it can be written in exponential form as

$$R_y(\theta) = e^{-i\theta Y/2}, \quad (2.69)$$

where Y denotes the Pauli- Y operator.

The rotation angle θ acts as a trainable parameter, making the R_y gate particularly suitable for PQCs and quantum machine learning (QML) models.

2.7.9.3 Pauli-Z Gate

The Pauli-Z gate is a single-qubit phase-flip operation defined by

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \quad (2.70)$$

Although it does not alter measurement probabilities in the computational basis, it modifies the relative phase between basis states, which is essential for controlling quantum interference effects.

2.7.9.4 Controlled-NOT (CNOT) Gate

The Controlled-NOT (CNOT) gate is a two-qubit unitary operation that flips the target qubit conditioned on the control qubit being in state $|1\rangle$.

In the computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, its matrix representation is

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (2.71)$$

The CNOT gate is essential for generating entanglement and, together with arbitrary single-qubit gates, forms a universal set for quantum computation.

In circuit diagrams (see Fig. 2.7), the filled dot denotes the control qubit, whereas the $\oplus$ symbol indicates the target qubit.

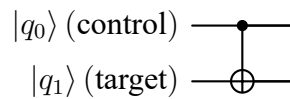


Figure 2.7: CNOT gate illustrating the control and target qubits.

2.8 From Classical Machine Learning to Quantum Machine Learning

In classical machine learning, a model is typically formulated as a parametric function

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}^m, \quad (2.72)$$

which maps an input vector to an output through a sequence of linear and nonlinear transformations. QML follows the same high-level learning principle but differs in the nature of the underlying computational model. Instead of classical functions, the core transformation is implemented by a quantum circuit acting on quantum states rather than real-valued vectors. The trainable parameters are no longer weights of matrix multiplications but continuous variables controlling quantum gates, such as rotation angles.

This shift in computational paradigm introduces two key conceptual differences compared to classical machine learning:

1. **Quantum data representation:** Data are processed as quantum states whose evolution is governed by unitary transformations. The expressive power of a QML model is therefore determined by the circuit structure and the entanglement it can generate, rather than solely by network depth or width.
2. **Probabilistic model outputs:** The output of a quantum model is inherently probabilistic, as it is obtained through measurement. Measurements yield expectation values or sampled outcomes rather than deterministic outputs.

Despite these differences, QML does not replace the classical learning framework but extends it. The training loop remains classical: measurement outcomes are used to compute a loss function $\mathcal{L}(\theta)$, gradients with respect to circuit parameters are evaluated (e.g., via the parameter-shift rule), and parameters are updated using standard optimization algorithms such as SGD.

In this sense, QML can be viewed as a generalization of classical machine learning in which the parametric model is implemented by a quantum process.

This perspective naturally leads to hybrid quantum–classical architectures, where quantum circuits are embedded as trainable components within classical learning pipelines.

2.9 QML Categories

QML can be classified into four categories according to the nature of the data and the type of computational processor employed. Specifically, the classification depends on whether the data

are classical (C) or quantum (Q), and whether the computation is performed on a classical or a quantum computer.

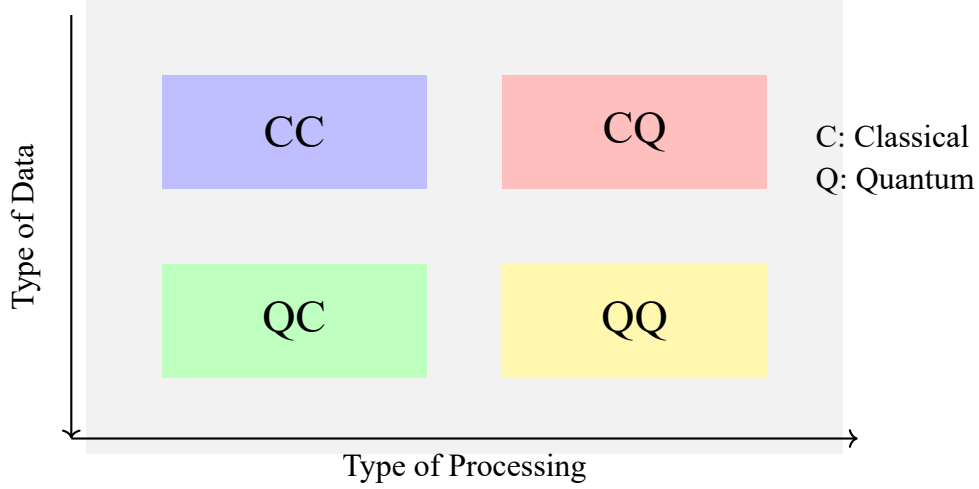


Figure 2.8: Categories of QML based on types of data and processor.

- **CC (Classical–Classical):** Refers to the processing of classical data on classical computers. The algorithms may be inspired by quantum computing ideas, but the implementation remains entirely classical.
- **CQ (Classical–Quantum):** Refers to the processing of classical data using quantum machine learning algorithms executed on quantum hardware.
- **QC (Quantum–Classical):** Describes the use of classical machine learning methods to analyze quantum data. It is an active research area, particularly for tasks such as qubit control, calibration, and readout.
- **QQ (Quantum–Quantum):** Concerns the processing of quantum data using quantum machine learning algorithms. This approach, often referred to as Fully Quantum Machine Learning (FQML), is primarily considered a long-term research direction as quantum hardware continues to advance.

2.10 Deep Quantum Neural Networks (DQNN)

A Deep Quantum NN (DQNN) is a quantum circuit composed of $L + 2$ layers of qubits. The first layer is referred to as the *input layer*, the last layer as the *output layer*, and the L intermediate layers as the *hidden layers*, indexed by l (see Fig. 2.9).

Given an input quantum state described by a density matrix ρ_{in} , the output state of a DQNN can be expressed as a composition of completely positive (CP) maps:

$$U^1 = U_3^1 U_2^1 U_1^1$$

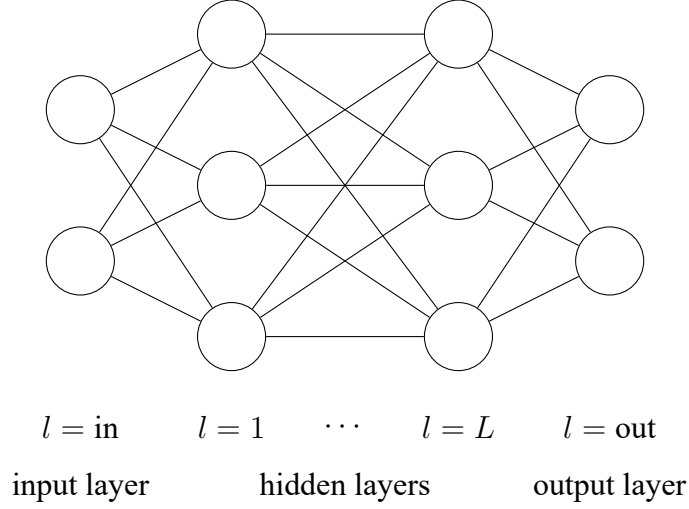


Figure 2.9: Deep Quantum NN (DQNN) architecture.

$$\rho_{\text{out}} = \mathcal{E}(\rho_{\text{in}}) = \mathcal{E}_{L+1} \left(\mathcal{E}_L \left(\dots \mathcal{E}_2 \left(\mathcal{E}_1(\rho_{\text{in}}) \right) \dots \right) \right). \quad (2.73)$$

Each map $\mathcal{E}_l$ corresponds to one network layer and is defined as:

$$\mathcal{E}_l(X_{l-1}) = \text{tr}_{l-1} \left(\prod_{j=1}^{m_l} U_j^l (X_{l-1} \otimes |0 \dots 0\rangle_l \langle 0 \dots 0|) \prod_{j=1}^{m_l} (U_j^l)^\dagger \right). \quad (2.74)$$

where:

- U_j^l is the unitary operator associated with the j -th quantum perceptron,
- m_l denotes the number of perceptrons between layers $l - 1$ and l ,
- tr_{l-1} denotes the partial trace over the previous layer.

In Table 2.1 we provide a visual explanation of the mathematics explained here

2.11 Parameterized Quantum Circuits (PQCs)

Parameterized Quantum Circuits (PQCs) are a class of quantum algorithms designed to leverage near-term quantum hardware effectively. In QML applications, a PQC is typically structured into two conceptual components. The first component is a data encoding circuit (also referred to as a feature map), which embeds classical input data into a quantum state. The parameters of this block depend solely on the input data and are not subject to training. The second component

is a variational circuit (ansatz), which contains trainable parameters optimized using a classical optimizer. This block constitutes the learnable part of the model [22].

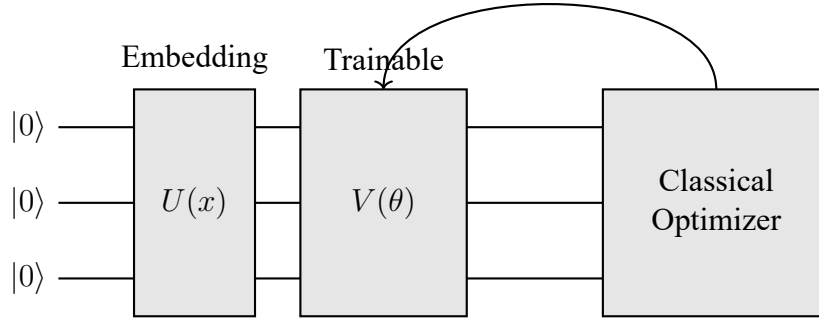


Figure 2.10: Hybrid classical–quantum (CQ) parameterized quantum circuit architecture.

2.12 Training a QNN / Computational Environment

This section, presents the basic information for the necessary understanding of the computational infrastructure’s definitions. These are required to conduct the experiments of this work and their practical utilization is described in detail in the Methodology section.

2.12.1 High-Performance Computing (HPC)

High-Performance Computing (HPC) refers to computing systems that utilize clusters of powerful processors operating in parallel to process large-scale datasets and solve computationally demanding problems at extremely high speeds [23]. HPC infrastructures enable scientific simulations, artificial intelligence applications, and compute-intensive workloads that exceed the capabilities of conventional systems.

2.12.2 SLURM Workload Manager

SLURM (Simple Linux Utility for Resource Management) is a workload manager responsible for allocating computational resources and scheduling jobs across HPC clusters [24]. A SLURM job submission script explicitly defines required resources such as CPU cores, memory allocation, GPU usage, execution time limits, and output handling. By declaring these parameters in advance, SLURM ensures efficient scheduling, resource fairness, and reproducibility of experiments.

2.12.3 CPU vs GPU Architecture

CPUs are optimized for sequential execution and complex control logic, whereas GPUs are designed for massively parallel computation. This architectural difference makes GPUs particularly effective for workloads dominated by matrix multiplications and tensor operations, such as deep learning. However, GPU performance is constrained by memory availability. Therefore, the selection between CPU and GPU resources depends on workload structure, memory requirements, and achievable parallelism.

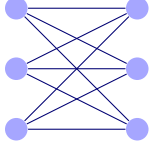
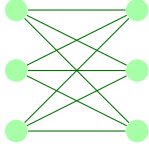
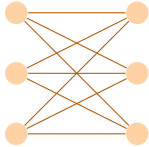
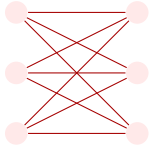
Stage	Description	Schematic
Initialisation	Randomly assign unitary perceptrons U_j^l for all layers l and perceptrons j . All qubits are initialised in $ 0\rangle$.	 Initialisation
Feed-forward	For each input state ρ_x^{in} , apply sequential CP maps $\rho_x^l = \mathcal{E}^l(\rho_x^{l-1})$. Apply $U^l = U_{m_l}^l \cdots U_1^l$ and trace out layer $(l - 1)$.	 Feed-forward
Back-propagation	Apply the adjoint channel $\mathcal{F}^{l\dagger}$. Tensor with identity on layer $(l - 1)$, apply $U^{l\dagger}$, and trace out layer l .	 Backprop
Update Rule	Compute update matrices K_j^l and update each unitary: $U_j^l \leftarrow e^{i\varepsilon K_j^l} U_j^l,$ preserving unitarity.	 Update

Table 2.1: Training algorithm of a Deep Quantum NN (DQNN).

3 Framework Analysis and System Models

In this chapter, we present and analyze the main models studied in this thesis. In particular, we describe the HAE and the CAE introduced in [3], as well as the UR-SWIPT autoencoder framework of [12], which considers a nonlinear communication setting.

3.1 CAE vs HAE over Fading Channels

3.1.1 Common Framework for the Autoencoders

The CAE and the HAE considered in this work share the same overall communication pipeline and differ only in the implementation of the transmitter. In both cases, the input message is selected from the set $\mathcal{M} = \{0, 1, \dots, M-1\}$, represented as an M -dimensional one-hot vector, and encoded into a transmitted signal spanning n channel uses.

The transmitted signal is represented in the complex baseband domain. More specifically, the encoded representation can be written as

$$x = x_{\text{Re}} + jx_{\text{Im}}, \quad (3.1)$$

where x_{Re} and x_{Im} denote the real and imaginary components, respectively. In the considered implementations, these two components are handled separately and combined to form the final transmitted signal representation. Before transmission, the encoded signal is normalized so that it satisfies the average power constraint.

After normalization, the transmitted signal passes through the channel layer, which introduces the corresponding propagation impairments. In this work, the common framework is evaluated under Rayleigh fading, Rician fading, and a 3GPP-based channel model.

At the receiver side, both systems use the same classical decoder. The decoder takes as input the stacked real and imaginary components of the received signal, forming a $2n$ -dimensional input vector. This representation is then mapped to the final M -dimensional output space, where a softmax layer produces the probability of each possible message. The estimated message is obtained by selecting the class with the highest output probability.

Therefore, the common part of the CAE and HAE lies in the message representation, signal normalization, channel model, and decoder structure, while the main difference between the

two architectures is the way the transmitter is implemented.

For the HAE, the encoder is implemented using PQCs, where the number of trainable parameters depends on the number of qubits n and the number of quantum layers L . The decoder architecture remains identical for both CAE and HAE.

3.1.2 Classical Encoder Architecture

In the CAE, the transmitter is implemented as a fully classical NN encoder. The input message is first converted to its M -dimensional one-hot representation and is then processed by two fully connected layers. The first layer maps the input to an intermediate M -dimensional representation, followed by a ReLU activation function, while the second layer maps the result to a final $2n$ -dimensional output vector. More specifically, the encoder operation can be written as

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_1 \mathbf{u}_s + \mathbf{b}_1), \quad (3.2)$$

$$\mathbf{z} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2, \quad (3.3)$$

where $\mathbf{u}_s$ denotes the one-hot representation of message s , $\mathbf{h} \in \mathbb{R}^M$ is the hidden representation, and $\mathbf{z} \in \mathbb{R}^{2n}$ is the encoder output.

After normalization, the first n components of the encoder output are interpreted as the real part of the transmitted signal, while the remaining n components are interpreted as the imaginary part.

Therefore, in the present implementation, the CAE learns the complete transmitted representation through a single classical encoder, while the real and imaginary signal paths are obtained by splitting the final encoder output into two equal parts. A detailed illustration of the CAE architecture is shown in Fig. 3.1.

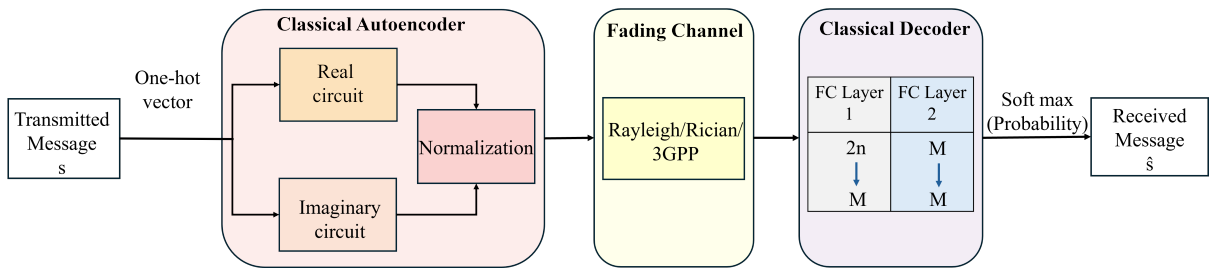


Figure 3.1: CAE architecture for E2E communication system.

3.1.3 Quantum Encoder Architecture

The HAE employs the same E2E communication framework as the CAE, with the encoder being the only modified component, working as a quantum circuit and called quantum encoder. In the following subsections we provide a detailed explanation of the system model. Fig 3.2 demonstrates the implemented HAE architecture.

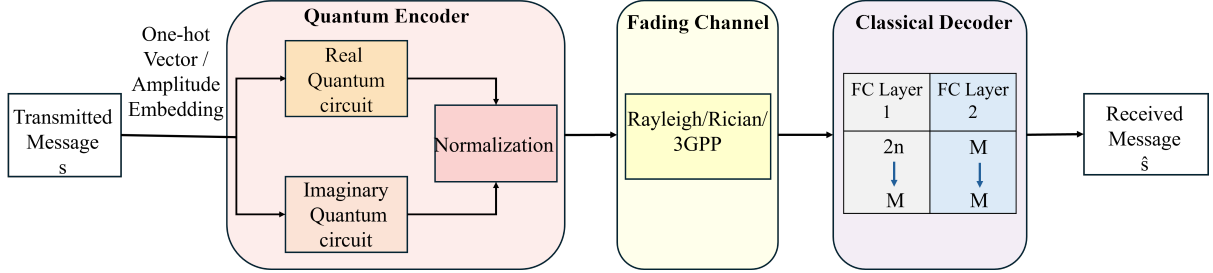


Figure 3.2: Hybrid quantum-classical autoencoder architecture with unified channel block.

3.1.3.1 Amplitude Embedding of the Classical Message

In contrast to the classical encoder, this one-hot representation is not directly mapped to channel symbols through FC layers, but is instead used to construct a quantum state that serves as the input to a PQC. This design enables the encoding process to exploit quantum feature mappings while preserving compatibility with the overall communication system. Each source message s , is first represented as an M -dimensional one-hot vector. This one-hot vector is then mapped to a quantum state using amplitude embedding, which converts the classical input into a computational basis state of an n -qubit quantum system. Given a real-valued input vector $\mathbf{x}_d \in \mathbb{R}^N$, with $N = 2^n$, amplitude embedding prepares the quantum state

$$|\psi\rangle = \frac{1}{\sqrt{\sum_{i=1}^N |x_i|^2}} \sum_{i=1}^N x_i |i\rangle, \quad (3.4)$$

where x_i is the i -th element of $\mathbf{x}_d$, and $|i\rangle$ denotes the i -th computational basis state.

When $n = k$, the one-hot vector is directly embedded into the quantum state space. When $n > k$, the additional $n - k$ qubits are initialized in the ground state $|0\rangle$, allowing the M -dimensional one-hot vector to be embedded into the higher-dimensional Hilbert space without loss of information. As a result, each input message is mapped to a unique and orthogonal quantum basis state.

This quantum state is then processed by the PQC, which learns an efficient representation for E2E communication.

Illustrative Example:

Table 3.1: Step-by-step amplitude embedding example for $k = 2$, $M = 4$, and $n = 3$ qubits.

Step	Value	Meaning
Information bits per message	$k = 2$	Each message is represented using two classical bits.
Number of possible symbols	$M = 2^k = 4$	The message set contains four possible symbols: $s \in \{0, 1, 2, 3\}$.
Number of qubits	$n = 3$	The quantum encoder uses three qubits, so the Hilbert space has dimension $2^n = 8$.
Input message	$s = 2$	The example considers the third symbol if zero-based indexing is used.
Binary representation	10	The classical message $s = 2$ is represented by the two-bit word 10.
Extra qubit initialization	$ 0\rangle$	Since $n > k$, one additional qubit is initialized in the ground state.
Shift amount	$n - k = 3 - 2 = 1$	Because the quantum state has one extra qubit, the message index must be shifted left by one bit.
Non-zero amplitude index	$s \ll 1 = 2 \ll 1 = 4$	The binary index changes from 10 to 100, so the non-zero amplitude is placed at index 4.
Computational basis state	$ 100\rangle$	Index 4 corresponds to the basis state $ 100\rangle$ in a three-qubit system.
Amplitude vector	$[0, 0, 0, 0, 1, 0, 0, 0]$	Only the amplitude of $ 100\rangle$ is non-zero; all other basis states have zero amplitude.

Information bits per message	$k = 2$
Number of possible symbols	$M = 2^k = 4$
Number of qubits	$n = 3$
Input message	$s = 2$
Binary representation	10
Non-zero amplitude index	$s \ll (n - k) = 2 \ll 1 = 4$
Computational basis state	$ 100\rangle$
Amplitude vector	$[0, 0, 0, 0, 1, 0, 0, 0]$
Extra qubit initialization	$ 0\rangle$ state

Table 3.2: Example of amplitude embedding for $k = 2$ and $n = 3$ qubits.

3.1.3.2 PQC Structure

Following the amplitude embedding stage, the encoded quantum state is processed by L identical PQC layers, where $L = 3$ in this work. The same PQC architecture is applied independently to both quantum circuits corresponding to the real and imaginary signal components.

Each PQC layer consists of a sequence of parameterized single-qubit rotation gates followed by entangling operations. Specifically, R_y rotation gates are applied to all qubits, with rotation angles scaled by π . These rotation angles represent the trainable parameters of the quantum model and play a role analogous to the weights in classical neural networks.

After the rotation stage, qubit entanglement is introduced using CNOT gates arranged in a nearest-neighbor configuration. In this setup, the i -th qubit acts as the control qubit, while the $(i + 1)$ -th qubit serves as the target qubit. This entangling structure enables correlations between adjacent qubits and enhances the expressive capability of the quantum circuit.

The output of each PQC layer is obtained through measurements in the Pauli- Z basis, producing classical expectation values. The outputs of the two parallel quantum circuits, corresponding to the real and imaginary components, are then stacked and normalized under a fixed power constraint, ensuring a consistent transmit power before passing through the channel layer.

Where:

- W_i : trainable weights
- $\bullet$: control unit
- $\circ$: target qubit

The whole PQC layer is implemented three times.

Four lines means $n = 4$, where n is the number of qubits.

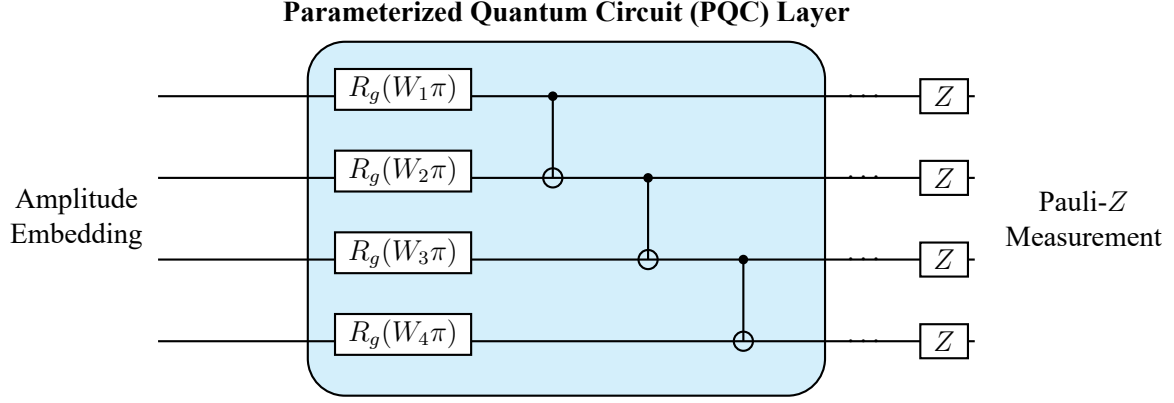


Figure 3.3: Structure of a single PQC layer with parameterized rotations, nearest-neighbor entanglement, and Pauli- Z measurements.

Mathematical Formulation of the PQC

Let n be the number of qubits, L the number of PQC layers (here $L = 3$), and let the amplitude embedding stage prepare a normalized quantum state

$$|\psi_0\rangle \in \mathbb{C}^{2^n}, \quad \|\psi_0\|_2 = 1.$$

1. PQC Layer Structure Each PQC layer $l \in \{1, \dots, L\}$ is defined as a unitary operator

$$U^{(l)}(\theta^{(l)}) = U_{\text{ent}} U_{\text{rot}}(\theta^{(l)}),$$

where $\theta^{(l)} = (\theta_1^{(l)}, \dots, \theta_n^{(l)})$ denotes the trainable parameters of layer l .

2. Single-Qubit Rotation Stage The rotation sub-layer applies parameterized R_y gates to all qubits:

$$U_{\text{rot}}(\theta^{(l)}) = \bigotimes_{i=1}^n R_y\left(\pi \theta_i^{(l)}\right),$$

with

$$R_y(\alpha) = \begin{pmatrix} \cos(\alpha/2) & -\sin(\alpha/2) \\ \sin(\alpha/2) & \cos(\alpha/2) \end{pmatrix}.$$

The parameters $\theta_i^{(l)} \in \mathbb{R}$ play a role analogous to trainable weights in classical NNs, controlling the quantum state transformation.

3. Entangling Stage Entanglement is introduced via nearest-neighbor CNOT gates, forming a linear chain:

$$U_{\text{ent}} = \prod_{i=1}^{n-1} \text{CNOT}_{i, i+1},$$

This entangling structure enables the circuit to model correlations between adjacent qubits, thereby enhancing the expressive capacity of the PQC.

4. Multi-Layer PQC Evolution After L PQC layers, the resulting quantum state is

$$|\psi_L\rangle = \left(\prod_{l=1}^L U^{(l)}(\theta^{(l)}) \right) |\psi_0\rangle.$$

5. Measurement in the Pauli-Z Basis The output of the PQC is obtained by measuring each qubit in the Pauli-Z basis. For the i -th qubit, the measurement outcome is the expectation value

$$z_i = \langle \psi_L | Z_i | \psi_L \rangle,$$

where

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Collecting all measurements yields the classical output vector

$$z = (z_1, z_2, \dots, z_n)^\top \in [-1, 1]^n.$$

6. Stacking and Power Normalization Let z_{Re} and z_{Im} denote the outputs of the PQCs corresponding to the real and imaginary branches, respectively. These outputs are stacked to form the transmitted symbol vector

$$x = \begin{bmatrix} z_{\text{Re}} \\ z_{\text{Im}} \end{bmatrix} \in \mathbb{R}^{2n}.$$

To enforce a fixed transmit power constraint, the vector is normalized as

$$\tilde{x} = \sqrt{P} \frac{x}{\|x\|_2},$$

where P denotes the desired average transmit power. The normalized vector $\tilde{x}$ is subsequently passed to the channel layer.

3.2 An AE-based Framework for Non-linear UR-SWIPT Channels

Unified Receiver Simultaneous Wireless Information and Power Transfer (UR-SWIPT) is a communication framework in which the same received RF signal is used jointly for information

decoding (ID) and energy harvesting (EH) through a unified receiver architecture. In contrast to conventional SWIPT receivers, where the received signal is typically split into separate EH and ID processing paths, the UR-SWIPT receiver relies on a common front-end, usually based on a rectifying circuit, in order to perform both tasks using the same hardware structure [12].

The main advantage of this architecture is its low hardware complexity and improved energy efficiency, which make it particularly attractive for low-power and battery-constrained wireless devices. However, this receiver structure also introduces a major modeling difference compared to conventional communication systems. Since the received signal is processed through a rectifier, the input-output relation is no longer linear. Instead, the effective channel becomes nonlinear, because the rectified output depends on the received waveform through a nonlinear device law rather than through a simple affine transformation.

As a result, the channel in UR-SWIPT cannot be modeled only through additive noise and linear fading. The rectifier itself becomes part of the effective communication channel, and the output used for decoding is determined by a nonlinear transformation of the transmitted signal. For this reason, UR-SWIPT is naturally treated as a nonlinear E2E communication problem, where the learned encoder must adapt not only to the noise conditions, but also to the nonlinear behavior of the rectifying receiver. Fig. 3.4 illustrates the UR-SWIPT circuit as implemented in [12].

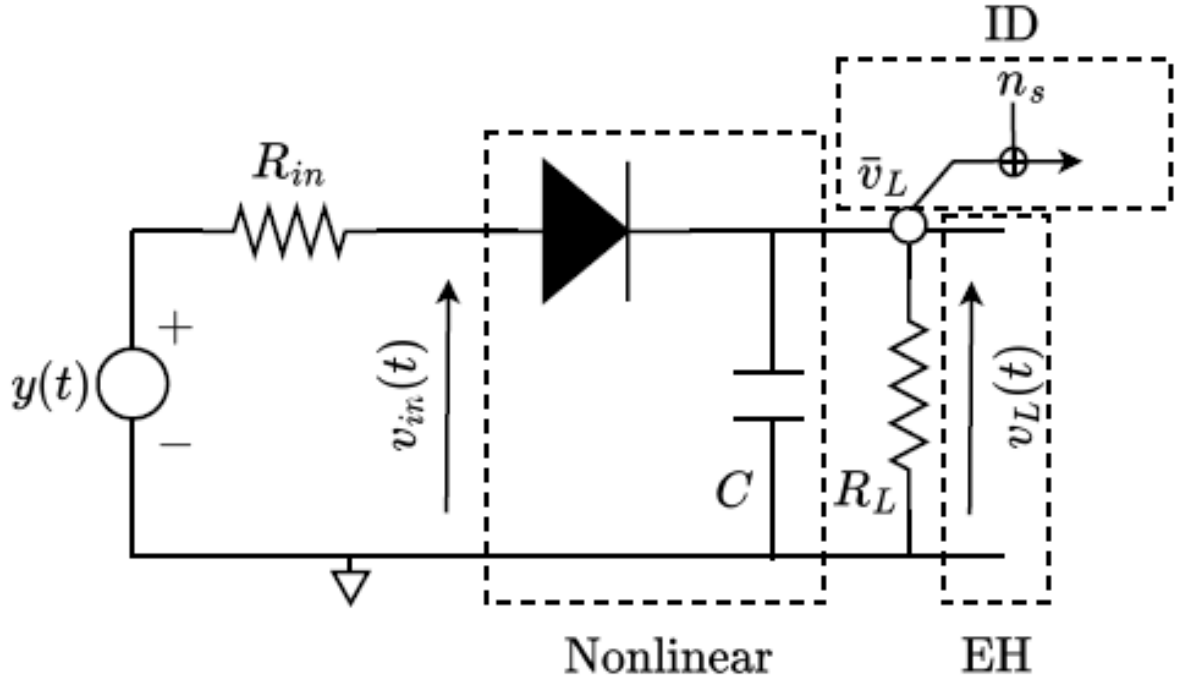


Figure 3.4: UR-SWIPT circuit.

3.2.1 System Model

In the considered UR-SWIPT setting, the source message $s \in \mathcal{M}$ is mapped by the encoder to a real-valued transmitted symbol $x = f_E(s)$, where the transmitted symbols belong to the constellation $\mathcal{X} = \{x_1, x_2, \dots, x_M\}$ with $x_m \in \mathbb{R}$.

The transmitted symbols are constrained by an average power condition, namely

$$\frac{1}{M} \sum_{m=1}^M x_m^2 \leq P_T, \quad (3.5)$$

where P_T denotes the transmit power.

Unlike the linear channel models considered previously, the transmitted symbol does not pass directly to the decoder through a linear noisy channel. Instead, it first enters the rectifying front-end of the unified receiver, which introduces the nonlinear transformation responsible for both energy harvesting and information decoding. Fig. 3.5 shows the block-level UR-SWIPT architecture adopted from [12].

To simplify the notation, we first define the scaled rectifier input as

$$z(x) = \frac{\sqrt{2P_T R_{\text{in}}}}{\eta V_T} x, \quad (3.6)$$

where R_{in} is the input resistance of the rectifier, V_T is the thermal voltage, and η is the diode ideality factor.

Following the adopted UR-SWIPT model, the deterministic rectifier output is expressed as

$$\bar{v}(x) = \eta V_T [W(K e^K I_0(z(x))) - K], \quad (3.7)$$

where $W(\cdot)$ denotes the Lambert- W function, $I_0(\cdot)$ is the modified Bessel function of the first kind, and

$$K = \frac{I_s R}{\eta V_T}, \quad (3.8)$$

with I_s being the reverse saturation current and R the load resistance.

After rectification, additive noise is introduced at the receiver, so that the signal used for information decoding is modeled as

$$y = \bar{v}(x) + w, \quad (3.9)$$

where $w \sim \mathcal{N}(0, \sigma^2)$.

The decoder then operates on the noisy rectified signal and produces the estimated message

$$\hat{s} = f_D(y). \quad (3.10)$$

Therefore, the complete E2E UR-SWIPT communication model can be written as

$$\hat{s} = f_D(\bar{v}(f_E(s)) + w). \quad (3.11)$$

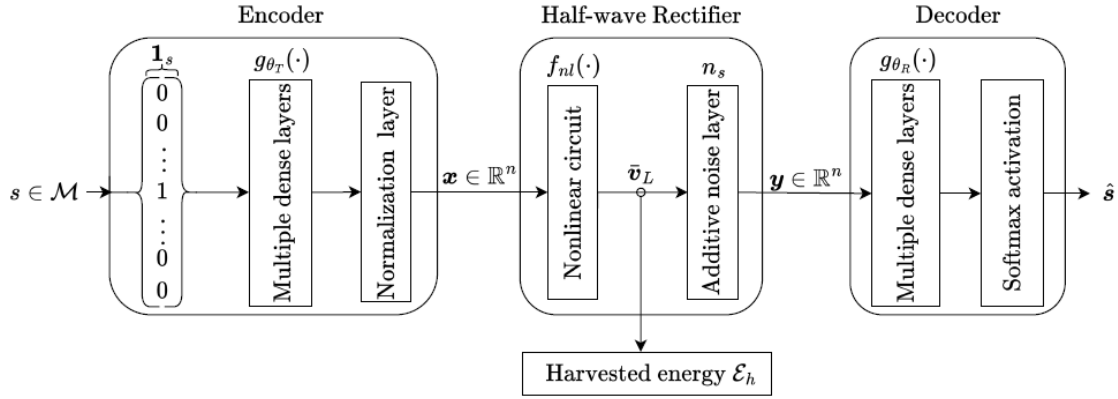


Figure 3.5: UR-SWIPT architecture.

3.2.2 Algorithmic Reference Model

In addition to the learned autoencoder-based solutions, the UR-SWIPT framework also considers an algorithmic reference model. Unlike the learned encoder, this approach does not optimize the transmitted constellation through training. Instead, the transmitted signal levels are constructed directly from the nonlinear rectifier model by using an analytical design procedure. More specifically, the algorithmic reference begins by defining uniformly spaced target voltage levels at the rectifier output,

$$v_m = m\Delta, \quad m = 0, 1, \dots, M-1, \quad (3.12)$$

where Δ is a spacing parameter. These target levels correspond to the desired rectified output values used for information decoding.

For each target voltage v_m , the corresponding transmitted amplitude y_m is obtained by inverting the nonlinear rectifier response, that is,

$$\bar{v}(y_m) = v_m. \quad (3.13)$$

In this way, the transmitted constellation is constructed so that, after passing through the non-linear rectifier, the received rectified levels are uniformly spaced.

The spacing parameter Δ is then adjusted so that the resulting transmitted constellation satisfies the same average power normalization as the learned encoder, namely

$$\frac{1}{M} \sum_{m=0}^{M-1} y_m^2 = 1. \quad (3.14)$$

Therefore, the algorithmic reference provides a model-based baseline that is derived directly from the known nonlinear channel characteristics. In contrast to the learned autoencoder, it does not adapt its constellation through data-driven optimization, but instead relies on explicit inversion and normalization of the rectifier response.

4 Modified Quantum Encoder Designs

In this chapter, we present and analyze the modifications introduced to the encoders in this work. In particular, the first two sections build upon the linear-channel CAE and HAE models presented in Chapter 3, whereas the final section is based on the nonlinear UR-SWIPT model introduced in the same chapter.

4.1 Lower-Parameter Quantum Encoder Variants

Having presented the baseline HAE architecture, our next objective is to reduce the number of trainable parameters in the quantum encoder. In the baseline $(7, 4)$ setting, the quantum encoder contains 42 trainable quantum parameters. We therefore examine whether this number can be reduced and, if so, how such parameter savings affect the overall system performance. For that reason, two low-parameter variants of the PQC layer were implemented and evaluated for this transmission setting. In both cases, the overall encoder structure remained unchanged. The only difference between the two implementations lies in the way the trainable rotation parameters are shared across qubits within each layer.

The first design corresponds to the 12-parameter model, denoted as P12 and shown in the figure 4.1 . In this variant, parameter sharing is based on qubit parity. More precisely, within each PQC layer, one trainable angle is assigned to all even-indexed qubits and a second trainable angle is assigned to all odd-indexed qubits. As a result, each layer uses only two trainable parameters per quantum branch. Since the encoder consists of three layers and two branches, corresponding to the real and imaginary parts, the total number of trainable quantum parameters becomes

$$N_{\text{p,enc}}^{12} = N_{\text{layers}} \times N_{\text{groups}} \times N_{\text{branches}} = 3 \times 2 \times 2 = 12.$$

This implementation represents the most compressed version of the PQC layer considered in this thesis, as it enforces strong weight sharing while preserving the same entangling topology and measurement process as the baseline architecture.

P12 encoder sharing rule

Two trainable groups per layer: even-indexed and odd-indexed qubits

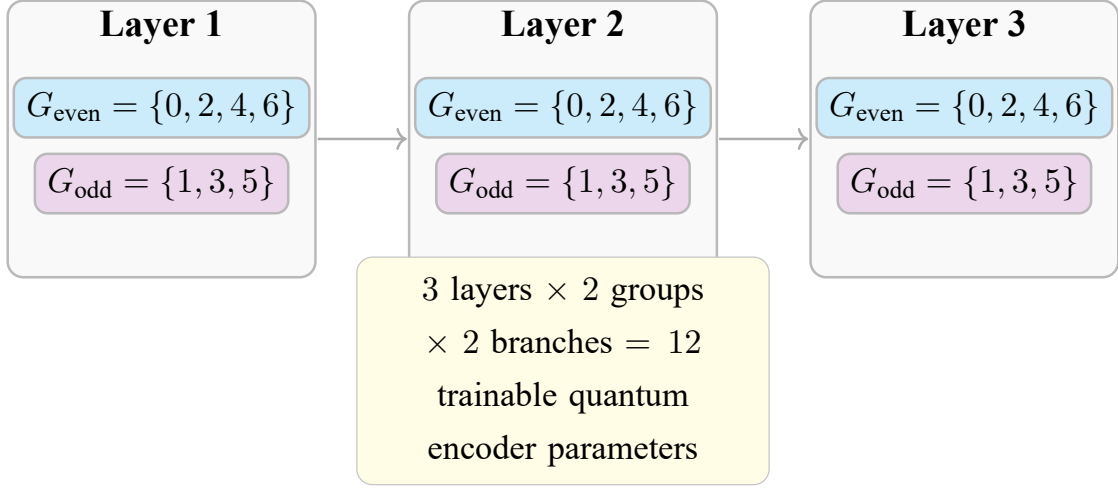


Figure 4.1: Schematic of the P12 low-parameter PQC variant. Within each layer, all even-indexed qubits share one trainable rotation parameter and all odd-indexed qubits share another.

The second design corresponds to the 18-parameter model, denoted as P18 and illustrated in figure 4.2. In this case, the parameter-sharing strategy is slightly relaxed in order to increase the capacity of the circuit while still maintaining a small parameter count. The seven qubits are divided into three fixed groups, namely

$$G_1 = \{0, 3, 6\}, \quad G_2 = \{1, 4\}, \quad G_3 = \{2, 5\}.$$

All qubits belonging to the same group share the same trainable rotation angle within a given layer. Consequently, each PQC layer requires three trainable parameters per branch instead of two. With three layers and two branches, the total number of trainable quantum parameters becomes

$$N_{\text{p,enc}}^{18} = N_{\text{layers}} \times N_{\text{groups}} \times N_{\text{branches}} = 3 \times 3 \times 2 = 18.$$

Compared to P12, this design provides a more flexible parametrization of the quantum transformations while still remaining parameter-efficient compared to a fully independent per-qubit parametrization.

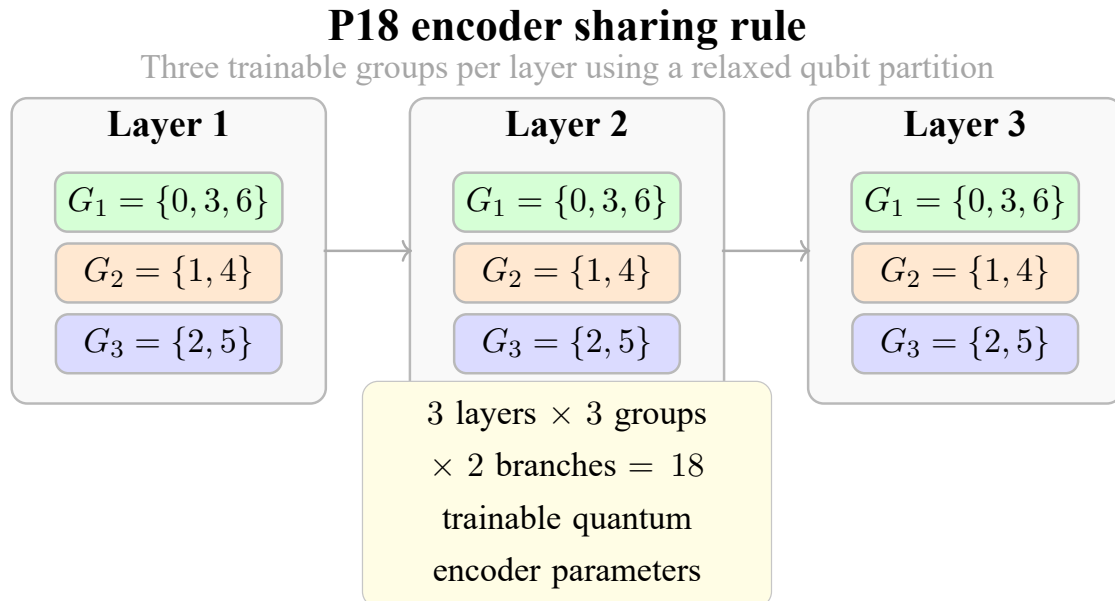


Figure 4.2: Schematic of the P18 low-parameter PQC variant. The qubits are divided into three fixed groups, providing more flexibility than P12 while still keeping the encoder compact.

4.2 Building a new Quantum Encoder from Scratch

Building upon the low-parameter PQC variants presented above, we next introduce a custom PQC design developed in this work. While the previous subsection focused on parameter-sharing strategies within the original encoder structure, the present discussion considers a modified circuit design.

The initial motivation behind this investigation was to further reduce the size of the PQC. However, since the ansatz adopted in [3] is already relatively compact, the potential benefit of further compression may be limited. For this reason, the present study also considers the opposite direction, namely the use of a larger ansatz. Instead of focusing on further compressing the quantum encoder, it may be more meaningful to slightly increase its capacity while limiting the complexity of the decoder. Such a modification could allow to release part of the model complexity from the classical decoder and reallocate it to the quantum encoder, thereby enabling a larger PQC design without increasing the total number of trainable parameters.

4.2.1 Decoder parameter reduction

In this setting, the proposed decoder replaces the original high-dimensional intermediate representation with a smaller hidden dimension. More specifically, the original decoder follows the

mapping

$$2n \rightarrow M \rightarrow M,$$

whereas the new decoder architecture is defined as

$$N_{\text{dec}} = (2n \rightarrow h \rightarrow M),$$

with

$$h = \min(2n, M).$$

The total number of trainable parameters in the original decoder is obtained by summing the parameters of its two fully connected layers. Therefore,

$$N_{\text{p,dec}} = N_{\text{p,dec}}^{(1)} + N_{\text{p,dec}}^{(2)} = (2nM + M) + (M^2 + M),$$

where $N_{\text{p,dec}}^{(i)}$ denotes the number of trainable parameters in the i -th layer.

Using the proposed parameter reduction, the total number of trainable parameters in the new decoder becomes

$$N'_{\text{p,dec}} = (2nh + h) + (hM + M).$$

For instance, for the $(7, 4)$ setting, where $n = 7$ and $M = 16$, the original decoder has the structure

$$14 \rightarrow 16 \rightarrow 16,$$

which corresponds to 512 trainable parameters. With the proposed rule,

$$h = \min(14, 16) = 14,$$

the new decoder becomes

$$N_{\text{dec}} = 14 \rightarrow 14 \rightarrow 16,$$

with 450 trainable parameters. Therefore, 62 parameters are released from the decoder and can be reallocated to the quantum encoder.

4.2.2 Custom PQC Ansatz Design

¹ In order to redesign the quantum encoder in a more structured way, the construction of the new ansatz follows the standard principles commonly used in QML architectures. More specifically,

¹In variational quantum circuits, an ansatz refers to a predefined circuit structure composed of a sequence of gates applied to selected qubits. Much like the architecture of a NN, it specifies the underlying form of the model, while the gate choices and their free parameters can be adjusted during the variational optimization process [25].

the ansatz is built from five main components: an embedding stage, a trainable variational block, an entangling structure, a layer repetition rule, and a final measurement stage [26].

The embedding stage is responsible for mapping the classical input to the quantum circuit. Common choices in the QML literature [27] include amplitude embedding, basis embedding, and angle embedding, with the selection typically depending on the structure and dimensionality of the input data.

The second component is the trainable variational block, which defines the learnable part of the circuit. This block contains the trainable quantum gates and determines how the ansatz adapts to the learning task. In general, such variational structures are formed by PQC rotations.

The third component is the entangling structure, which defines how information is shared across qubits. This part is essential, since it enables the circuit to capture correlations that cannot be modeled through independent single-qubit operations alone. Typical choices include nearest-neighbor CNOT chains, ring entanglement, CZ gates, and Ising-type couplings.

The fourth component is the layer repetition rule. Rather than designing each part of the circuit independently, a structured block is typically repeated multiple times. In this way, the depth of the ansatz can be controlled in a systematic manner, while preserving a clear and interpretable circuit structure.

Finally, the measurement stage converts the final quantum state into classical outputs. Depending on the design, this readout may be based on Pauli- Z expectation values, probabilities, or other observables. In the present framework, the measurement stage provides the classical outputs required to construct the real and imaginary branches of the encoder.

4.2.2.1 Embedding

In the proposed encoder design, the classical message is embedded into the quantum circuit using angle embedding with rotation gates instead of amplitude embedding. Each transmitted symbol is first converted into its k -bit binary representation. The binary values are then mapped to rotation angles according to a simple deterministic rule, where bit value 0 is mapped to angle 0 and bit value 1 is mapped to angle π .

In other words, let the transmitted message $s \in \mathcal{M}$ be represented by its k -bit binary vector

$$\mathbf{b}(s) = [b_1, b_2, \dots, b_k], \quad b_i \in \{0, 1\}. \quad (4.1)$$

The corresponding rotation angles are then defined as

$$\theta_i = \pi b_i, \quad i = 1, 2, \dots, k, \quad (4.2)$$

so that $b_i = 0$ gives $\theta_i = 0$, while $b_i = 1$ gives $\theta_i = \pi$.

Therefore, the embedded input state can be written as

$$|\psi_0(s)\rangle = \left(\bigotimes_{i=1}^k R_x(\theta_i) \right) |0\rangle^{\otimes k}. \quad (4.3)$$

This bit-to-angle mapping was selected in order to preserve the binary structure of the transmitted message while introducing a gate-based classical-to-quantum encoding mechanism. In contrast to amplitude embedding, which prepares the full quantum state directly from the input vector, the adopted approach injects the message information through R_x rotations.

Since the considered system involves $n > k$, the resulting bit-angle pattern is repeated cyclically until all n qubits are assigned an input angle. Therefore, the same embedding rule can be applied consistently across different (n, k) configurations without introducing additional trainable classical preprocessing.

4.2.2.2 Measurement Stage

Before fixing a measurement strategy, we examined whether one of the standard Pauli observables could be justified as our choice for the encoder and from then to continue on the other parts of constructing the PQC design. However, for the present wireless communication objective, no sufficiently strong task-specific argument was identified to support choosing a single fixed observable, such as Pauli- X , Pauli- Y , or Pauli- Z , from the outset. This is important because, in the present setting, the quantum encoder is not used as a final classifier. Its role is to generate useful latent signal features that will later pass through the channel and the classical decoder. For this reason, the measurement stage should not be treated as a secondary readout detail, but rather as an active component of the encoder design itself [11].

4.2.3 Implemented Encoder Design

The proposed encoder processes the binary input through two main quantum blocks, each responsible for a different part of the feature-generation procedure. The first block is the angle-embedding stage, where the input bits are mapped to rotation angles and encoded into the quantum circuit through R_x rotations. After this stage, the embedded state is processed by the three-layer PQC composed in [3].

The second block is a lightweight classical controller, which also takes the binary bits as input. The controller produces two outputs. The first output is a set of layer-dependent coefficients,

$$\gamma(s) = [\gamma_1(s), \gamma_2(s), \gamma_3(s)]^\top = \tanh(W_{\text{rot}}\mathbf{b}(s) + \mathbf{c}_{\text{rot}}), \quad (4.4)$$

with one coefficient assigned to each PQC layer. These coefficients are used to scale the trainable R_y rotations of the corresponding layer. More specifically, the layer scaling factor is defined as

$$\lambda_l(s) = 1 + \alpha \gamma_l(s), \quad (4.5)$$

where α is a fixed modulation constant. In the present implementation, $\alpha = 0.2$.

Therefore, if $\theta_{l,i}$ denotes the base trainable parameter of qubit i at layer l , then the effective rotation angle becomes

$$\theta_{l,i}^{\text{eff}}(s) = \pi \theta_{l,i} \lambda_l(s) = \pi \theta_{l,i} (1 + \alpha \gamma_l(s)). \quad (4.6)$$

In this way, the same trainable circuit is reused for all symbols, while its layer strength is adjusted according to the transmitted message.

The second controller output is a scalar readout angle,

$$\phi(s) = \frac{\pi}{2} \tanh(W_{\text{meas}} \mathbf{b}(s) + c_{\text{meas}}). \quad (4.7)$$

This value is applied as an additional R_y rotation before the final measurement stage. Therefore, although the physical measurement remains a Pauli- Z expectation value, the readout is first modified by a symbol-dependent rotation.

This same mechanism is applied independently to the two branches of the encoder, the real and the imaginary branch. Each branch has its own trainable quantum weights, its own controller-generated layer coefficients, and its own controller-generated readout angle. Finally, the outputs of the two branches are collected as expectation values, stacked into a single vector, and normalized so that the power constraint of the communication system is satisfied. The full block diagram of the architecture is given by Fig. 4.3

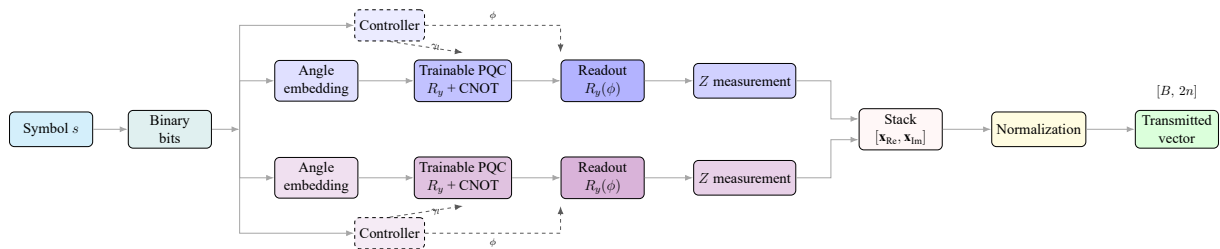


Figure 4.3: Structure of the implemented custom quantum encoder. The binary representation of the input symbol is used both for angle embedding and for the generation of control signals that adapt the variational layers and the readout stage of each branch.

4.2.4 Parameter Budget Control

The adaptive part of the proposed encoder is introduced under the parameter-saving margin created by the compact decoder. In general, a fully connected layer with input dimension a and output dimension b contains $N(a, b) = ab + b$ trainable parameters, where the first term corresponds to the weights and the second term to the biases.

Using this rule, each controller that maps k input bits to L layer coefficients contributes

$$N_{\text{rot}} = kL + L \quad (4.8)$$

trainable parameters, while each controller that maps k input bits to a single readout angle contributes

$$N_{\text{meas}} = k + 1. \quad (4.9)$$

Since the encoder contains two rotation controllers and two measurement controllers, the total number of additional controller parameters is

$$N_{\text{ctrl}} = 2(kL + L) + 2(k + 1). \quad (4.10)$$

For the $(7, 4)$ setting, where $k = 4$ and $L = 3$, this becomes $N_{\text{ctrl}} = 2(4 \cdot 3 + 3) + 2(4 + 1) = 2(15) + 2(5) = 40$.

Therefore, the adaptive encoder introduces 40 additional trainable parameters. Since the compact decoder releases 62 trainable parameters, the complete model still remains 22 parameters below the previous design budget. In this way, the encoder becomes more flexible while preserving the overall parameter-efficiency objective of the architecture.

4.3 Hybrid UR-SWIPT

In the hybrid implementation considered in this thesis, the encoder is replaced by a quantum-based architecture, while the nonlinear rectifier and the decoder remain in the classical domain. The role of the encoder is to produce transmitted symbols that remain distinguishable after passing through the nonlinear transformation $\bar{v}(\cdot)$, so that reliable decoding can still be achieved at the receiver.

The proposed model is based on the HAE framework introduced in Chapter 3, but with two important modifications for the UR-SWIPT setting. First, the PQC is implemented with $L = 3$ layers and $n = 3$ qubits for the ID task. Second, the encoder output is no longer treated as a $2n$ -dimensional transmitted representation with separately normalized real and imaginary parts.

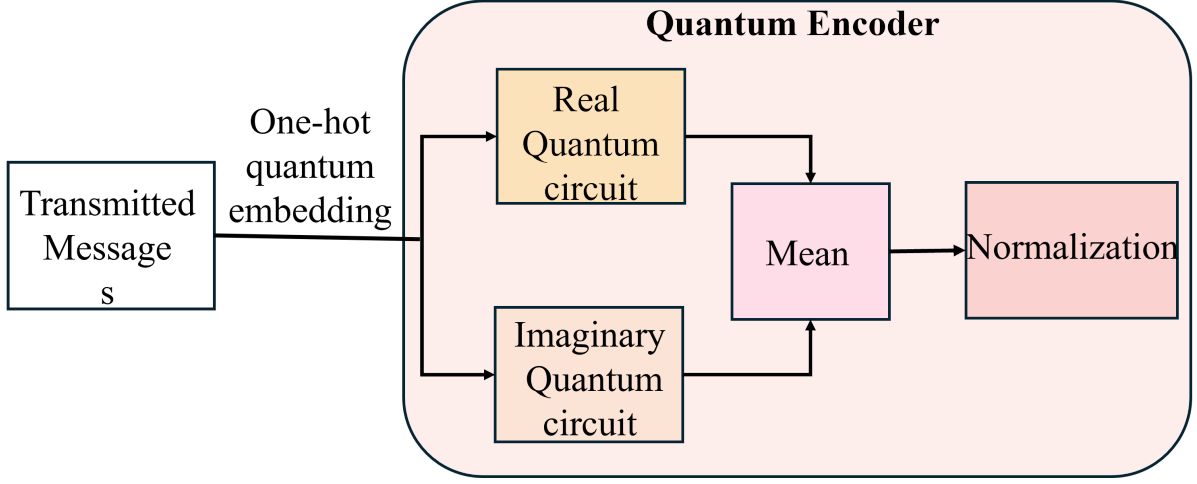


Figure 4.4: Hybrid quantum encoder architecture used in the UR-SWIPT framework.

Instead, the real and imaginary quantum branches each produce a single scalar output, denoted by x_{Re} and x_{Im} , respectively. These two outputs are then combined through their magnitude according to

$$x = \sqrt{x_{\text{Re}}^2 + x_{\text{Im}}^2}. \quad (4.11)$$

The resulting scalar is finally normalized before transmission through the nonlinear channel. If x denotes the batch of transmitted magnitudes, the normalized transmitted signal is given by

$$\tilde{x} = \frac{x}{\sqrt{\mathbb{E}[x^2]}}. \quad (4.12)$$

Therefore, although the encoder still preserves the two-branch quantum structure, its final output rule is modified in order to match the scalar-input nature of the UR-SWIPT rectifying receiver. The resulting encoder structure used in the UR-SWIPT framework is illustrated in Fig. 4.4.

In the next chapter, we evaluate the ID performance of the UR-SWIPT system under nonlinear channel conditions using this hybrid encoder and compare the obtained results with the corresponding classical autoencoder baseline reported in [12].

5 Numerical Results and Discussion

This section investigates the performance of the considered HAE and CAE frameworks. Their performance is evaluated using BLER as the primary performance metric. To characterize the robustness of the system under different channel conditions, the BLER performance is evaluated as a function of the SNR. In addition to BLER versus SNR performance, the training convergence behavior of the models is analyzed by monitoring the BLER evolution as a function of the training epochs. We note that, the models are trained at a fixed SNR value and subsequently tested over a wide range of SNRs to assess their generalization capability across different noise levels.

The second part of the evaluation focuses on the comparison of the trainable parameters between the CAE and the HAE, used to quantify the model complexity of each architecture. The total number of trainable parameters is computed by summing all learnable weights and biases in the NN components, as well as the trainable rotation angles of the PQCs in the HAE. For the CAE, the parameter count is determined by the fully connected layers of both the encoder and the decoder, where the input dimension is defined by the message space size M .

5.1 Linear Channel (HAE vs CAE)

5.1.1 Parameters

As specified in the reference study [3], a common set of parameters is adopted for both the HAE and CAE architectures in order to ensure a fair comparison. Specifically, the batch size is set to 32 and the models are trained for 80 epochs at a fixed training SNR of 10 dB, using the Adam optimizer with a learning rate of 0.001. Performance evaluation follows the same protocol as in the reference study and is conducted over a SNR range from 0 to 20 dB.

Apart from the parameters specified in the reference study, several additional implementation parameters had to be assumed in this work, since they were not explicitly reported in the paper. Training is performed using 1,000 steps per epoch, while performance evaluation is carried out using Monte Carlo simulations. For each SNR ratio point, the BLER is estimated over 75,000 independently generated test samples to ensure statistically reliable results. Regarding the channel configuration, the Rician fading model is parameterized by a K-factor = 5. The 3GPP-based channel follows the spatial channel model (SCM) [28].

Table 5.1: Comparison of parameters size between CAE and HAE architectures for different (n, k) configurations.

Scheme	(4,4)	(7,4)	(8,8)
CAE	1232	1532	209952
HAE	440	554	70192

5.1.2 Parameter savings

Before proceeding to the BLER vs SNR performance comparisons between the HAE and CAE architectures, it is important to highlight the parameter efficiency of the proposed approach. As discussed in [3], HAE can achieve significant reductions in the number of trainable parameters compared to purely classical architecture. This behavior is also verified in our implementation. As shown in Table 5.1, the HAE architecture consistently requires significantly fewer parameters compared to the CAE across all examined configurations. Specifically, for the (4,4), (7,4), and (8,8) settings, the number of parameters is reduced from 1232 to 440, from 1532 to 554, and from 209952 to 70192, respectively. This corresponds to a reduction of approximately 60%–70%, with the most pronounced gain observed for larger system dimensions.¹

5.1.3 BLER vs SNR Performance

We now demonstrate reliability performance in terms of BLER vs SNR for the (4,4), (7,4), and (8,8) settings, over 3 different fading channels.

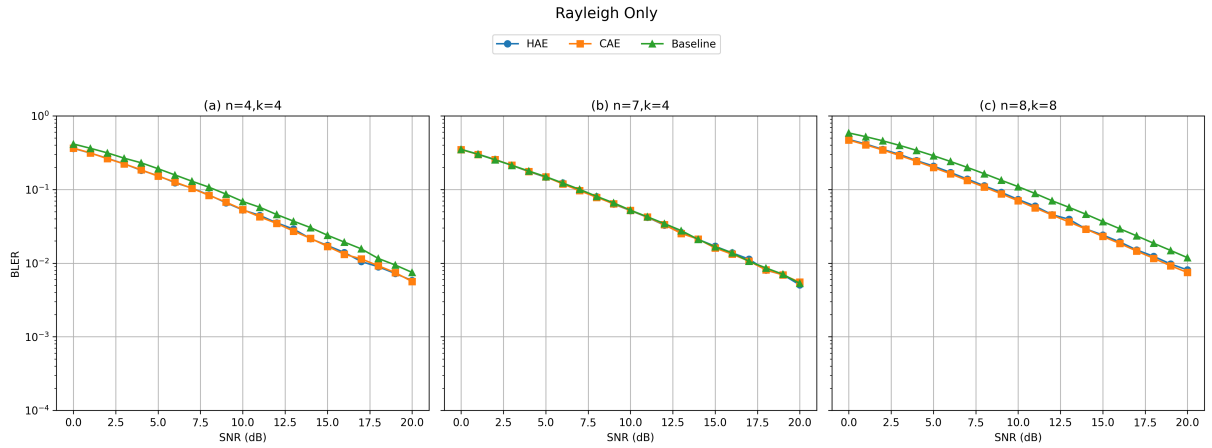


Figure 5.1: Rayleigh-channel comparison among HAE, CAE, and the corresponding baseline references for the (4, 4), (7, 4), and (8, 8) configurations. Here, the baseline refers to BPSK for the (4, 4) and (8, 8) settings, and to the Hamming code for the (7, 4) setting.

For the Rayleigh channel, the HAE remains very close to the CAE in the (4, 4) and (7, 4) cases.

¹We want to emphasize that these parameter savings do not necessarily translate to reduced computational complexity when implemented on classical hardware, since quantum circuits are typically simulated.

At 20 dB, the HAE achieves a BLER of approximately 5.8×10^{-3} for (4, 4) and 5.06×10^{-3} for (7, 4), compared to 5.62×10^{-3} and 5.52×10^{-3} for the CAE, respectively. This indicates that in the low- and medium-dimensional settings the hybrid encoder preserves almost the same reliability as the classical architecture while maintaining a substantially smaller parameter count. In the (8, 8) case, the gap becomes slightly larger, with the HAE reaching approximately 8.08×10^{-3} and the CAE 7.46×10^{-3} at 20 dB, but the two curves still remain close overall.

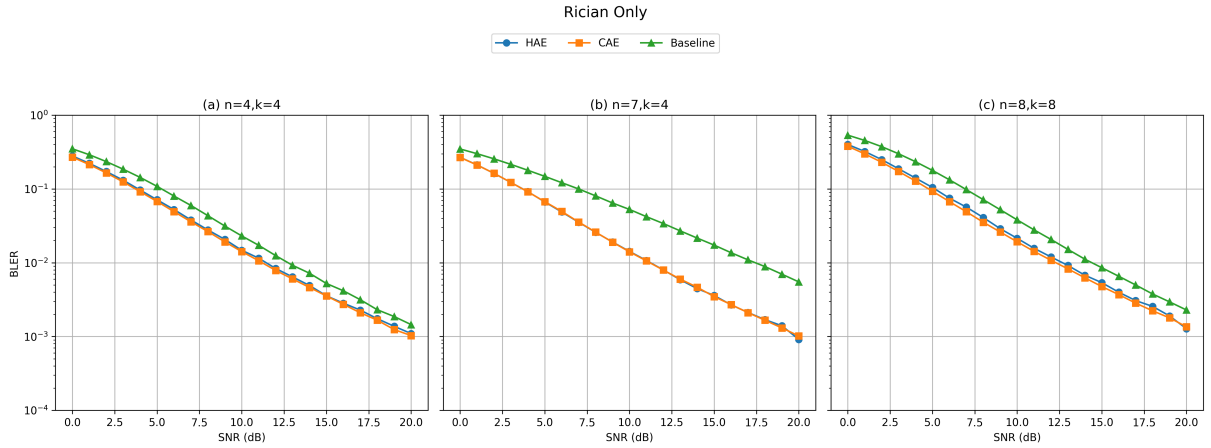


Figure 5.2: Rician-channel comparison among HAE, CAE, and the corresponding baseline references for the (4, 4), (7, 4), and (8, 8) configurations. Here, the baseline refers to BPSK for the (4, 4) and (8, 8) settings, and to the Hamming code for the (7, 4) setting.

A similar trend is observed in the Rician case. The HAE also remain very close to the corresponding CAE results. At 20 dB, the recorded BLER values are approximately 1.10×10^{-3} for (4, 4), 9.12×10^{-4} for (7, 4), and 1.28×10^{-3} for (8, 8). The (7, 4) configuration is especially notable, since the HAE slightly outperforms the CAE at the high-SNR end.

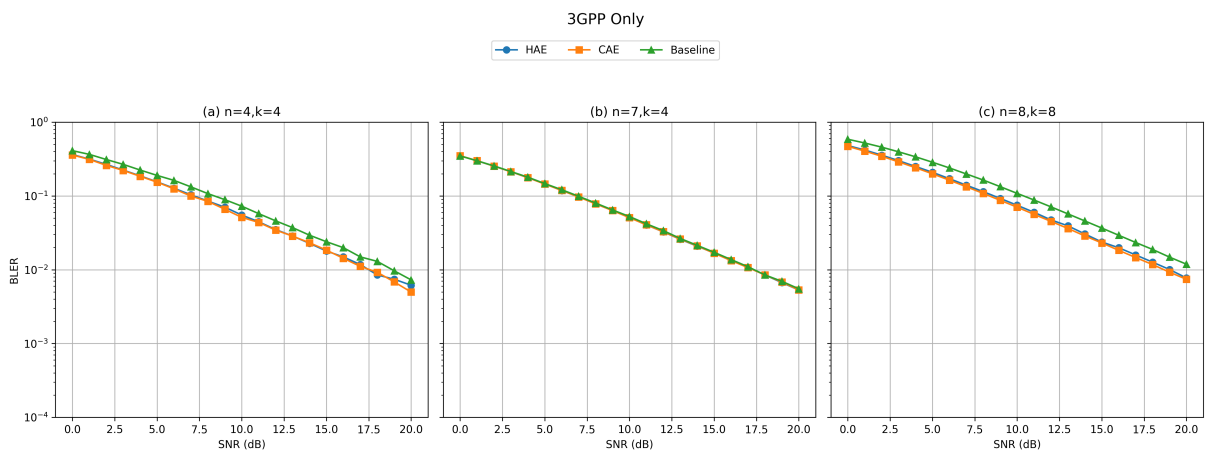


Figure 5.3: 3GPP-channel comparison among HAE, CAE, and the corresponding baseline references for the (4, 4), (7, 4), and (8, 8) configurations. Here, the baseline refers to BPSK for the (4, 4) and (8, 8) settings, and to the Hamming code for the (7, 4) setting.

Table 5.2: Parameter comparison for the (7, 4) original quantum encode and its low-parameter PQC variants.

Model Variant	Quantum Encoder Params	Decoder Params	Total Params
Original Quant Enc	42	512	554
Low-parameter PQC (P12)	12	512	524
Low-parameter PQC (P18)	18	512	530

For the 3GPP channel, the same general conclusion still holds. The HAE and CAE exhibit very similar BLER behavior, particularly for the (7, 4) setting, where both models practically overlap at high SNR, with values around 5.31×10^{-3} at 20 dB. For (4, 4) and (8, 8), the CAE remains slightly better, but the gap is still limited.

Overall, the present results are in agreement with the findings of [3]. In both the reference paper and our implementation, the hybrid architecture does not necessarily dominate the CAE in every setting, but it consistently achieves comparable BLER with a much smaller number of trainable parameters. The main difference is that our implementation shows small deviations across some channel cases, which is reasonable due to different implementation settings. Nevertheless, the HAE offers a favorable trade-off between parameter efficiency and communication reliability.

5.2 Lower-Parameter Quantum Encoder Variants

After we have analysed the setup in the previous chapter, in this section we provide a table 5.2 illustrating the parameters in the setup (7,4) for each architecture. Although the original setup was already compact in terms of the trainable parameters of the quantum encoder, we were able to reduce this number even further. The two reduced variants, P12 and P18, are compared below with the original HAE over the same Rayleigh channel.

Firstly, for the P12 setup, Fig. 5.4 clearly shows that the P12 architecture exhibits a higher BLER across the entire SNR range, with the performance gap becoming more noticeable at higher SNR values. At 20 dB, the 12P exhibits a BLER on the order of 0.007, which is noticeably worse than that of the original quantum encoder with the 42 trainable parameters which achieves a BLER of approximately 0.0055 at the same SNR point.

Next for the P18 setup, the corresponding BLER versus SNR comparison is shown in Fig. 5.5. In contrast to the P12 case, the P18 variant remains much closer to the original HAE performance over the entire Rayleigh range. This indicates that reducing the encoder to 18 trainable parameters preserves most of the performance while still offering a more compact model. Therefore, the P18 variant becomes a meaningful candidate for a broader comparison across different channel conditions. For this reason, we also extend the comparison to the full set of examined channels, shown in Fig. 5.6 in order to evaluate whether the same behavior is preserved beyond

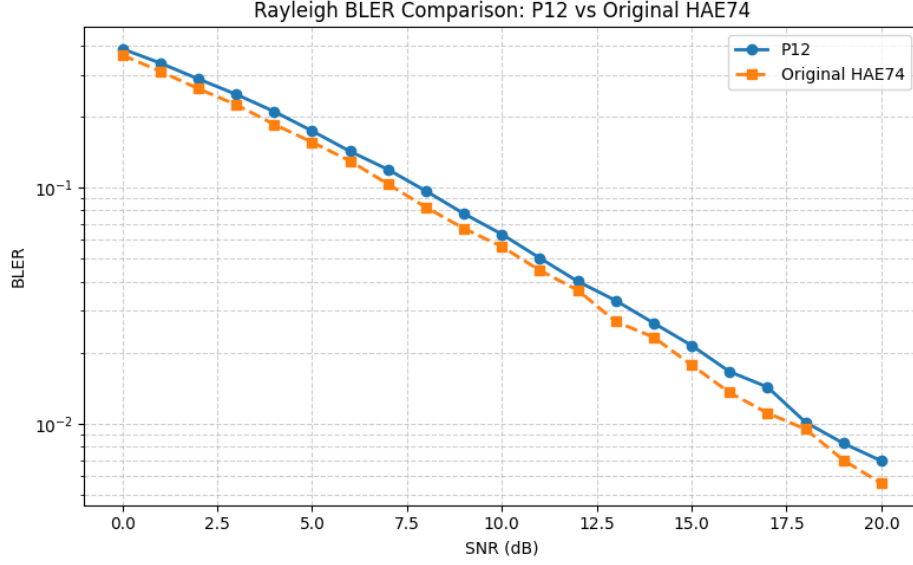


Figure 5.4: BLER versus SNR comparison between the original HAE(7,4) model and the low-parameter P12 variant over the Rayleigh channel.

the Rayleigh case.

In the 3GPP and Rayleigh cases, the two curves are very close over the entire SNR range, which shows that the lower-parameter encoder preserves almost the same behavior as the original HAE74. The observed degradation is small and remains consistent, indicating that most of the useful representation power is retained even with only 18 trainable quantum parameters. For the Rician channel, the difference becomes more visible, with the original HAE74 achieving lower BLER values across the full SNR range. Even so, the P18 model still follows the same overall trend and maintains acceptable performance, which suggests that the reduced architecture generalizes reasonably well across different propagation environments.

5.3 Results of the new Quantum Encoder

5.3.1 BLER vs Epoch

The first evaluation of the custom quantum encoder is presented in Fig. 5.7, where the BLER is monitored during training. The curve shows a clear reduction during the first epochs, moving from approximately 7.2×10^{-2} to a lower and more stable region close to 5.3×10^{-2} . After the initial decrease, the curve continues to fluctuate slightly, which is expected due to the stochastic nature of training and the limited size of the evaluated batches. Nevertheless, the overall behavior indicates that the model is able to learn a useful representation and improve its reliability during training. For comparison, Fig. 5.8 presents the corresponding BLER versus

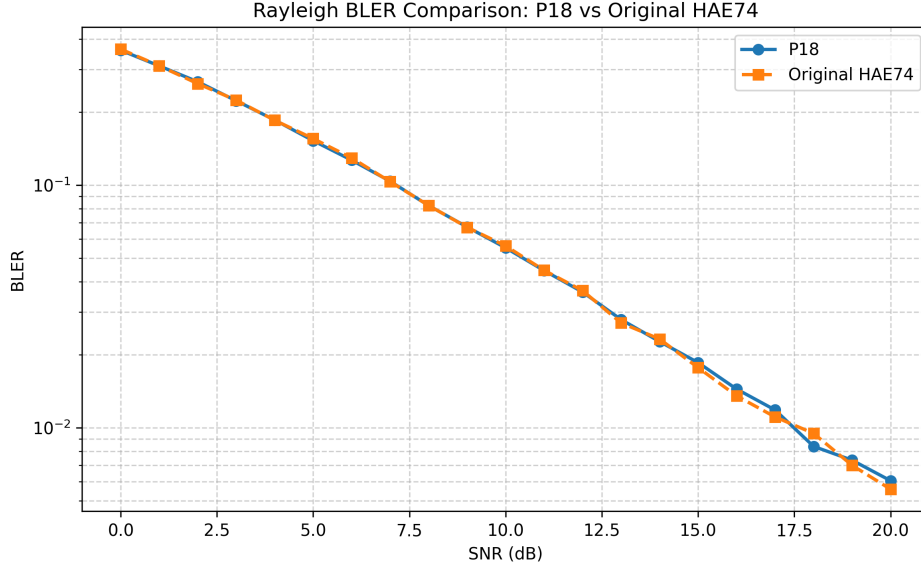


Figure 5.5: BLER versus SNR comparison between the original HAE(7,4) model and the low-parameter P18 variant over the Rayleigh channel.

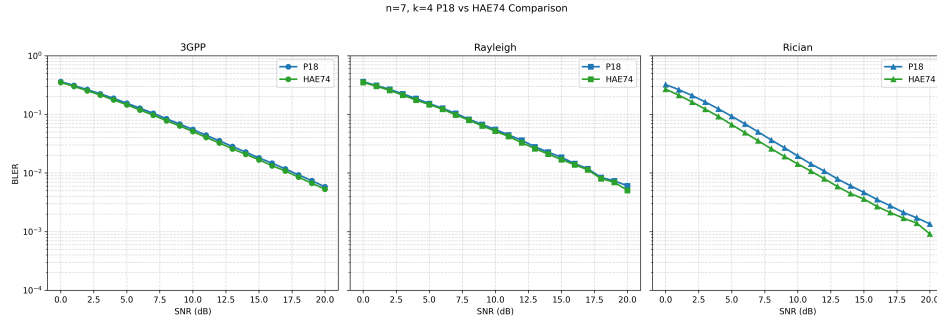


Figure 5.6: BLER versus SNR comparison between the original HAE(7,4) model and the low-parameter P18 variant across the full evaluation setting.

epoch behavior reported in the reference implementation of [3]. This comparison shows that the custom encoder reaches a BLER level very close to the one reported in the reference paper. More specifically, our curve starts at about 7.2×10^{-2} , falls below 5.6×10^{-2} during the first 10 epochs, and then remains roughly in the range of 5.3×10^{-2} to 5.5×10^{-2} . In Zhang et al., the AE and QAE curves also settle in approximately the same BLER region after the early training stage. This shows that, despite its different design, the proposed encoder achieves a final training behavior that is numerically close to the reference implementation and confirms the validity of the proposed training procedure.

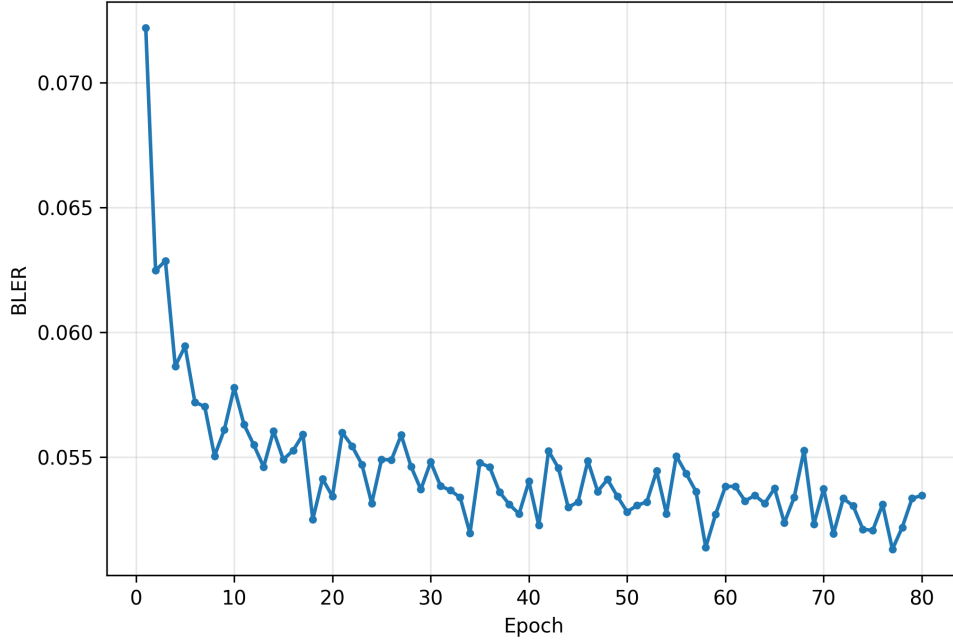


Figure 5.7: Training behavior of the custom quantum encoder, showing BLER as a function of the training epoch.

5.3.2 BLER over the Rayleigh channel

The following results are for the $(7, 4)$ setting and the considered channel model is Rayleigh fading. A comparison between Fig. 5.9 and Fig. 5.5 shows that the proposed custom encoder follows the same overall BLER trend as the reference $(7, 4)$ Rayleigh implementation. In both cases, the BLER starts at a relatively high level close to 2.6×10^{-1} at 0 dB and decreases steadily as the SNR increases. Around the mid-SNR region, the performance remains in the same range, with BLER values close to 5×10^{-2} at 10 dB and around 2×10^{-2} at 15 dB. At high SNR values, both curves continue to drop to the order of 10^{-2} and below, reaching approximately 6×10^{-3} at 20 dB. Therefore, the custom encoder achieves a BLER behavior that is numerically very close to the reference $(7, 4)$ Rayleigh result, which suggests that the new design preserves the main reliability characteristics of the original implementation across the full examined SNR range.

5.4 Non-Linear Channel (UR-SWIPT)

5.4.1 Parameter saving

In this subsection we plan to highlight the parameter efficiency of the two architectures in the non-linear channel setting. Similar to the linear-channel case, the HAE requires fewer trainable parameters than the CAE. This behavior is also confirmed in our UR-SWIPT implementation.

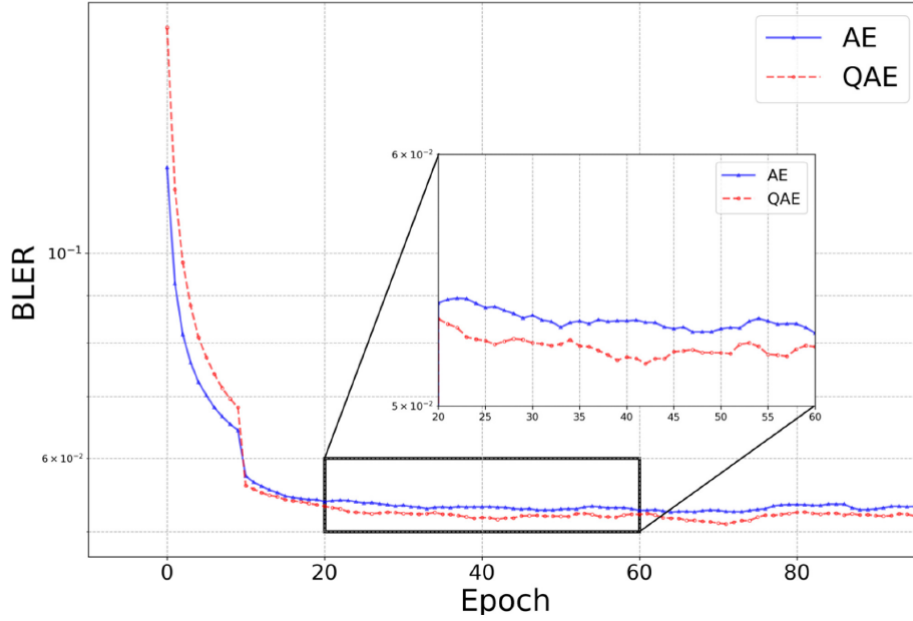


Figure 5.8: BLER as a function of the training epoch for the reference hybrid model reported by Zhang et al.

Table 5.3: Parameter comparison between CAE and HAE for the UR-SWIPT ID model in the $M = 8$, $P_t = -2.5$ dBm setup.

Scheme	Encoder	Decoder	Total
CAE	81	88	169
HAE	18	88	106

As shown in Table 5.3, for the $M = 8$ configuration, the CAE architecture contains 169 trainable parameters, whereas the HAE architecture contains only 106. The decoder size remains the same in both schemes, while the main reduction comes from the encoder. More specifically, the classical encoder requires 81 trainable parameters, whereas the hybrid encoder requires only 18 trainable quantum parameters. Overall, this corresponds to a parameter reduction of approximately 37.3%.

5.4.2 Performance Evaluation

The SER performance of the UR-SWIPT system is shown in Fig. 5.10. The comparison includes the algorithmic reference, the learned classical model, and the proposed HAE implementation. Across the examined SNR range, the three curves follow almost the same behavior, which indicates that the HAE is able to preserve the main detection performance of the learned UR-SWIPT model while using fewer trainable parameters.

At low SNR values, all methods have a relatively high SER, as the received signal is strongly

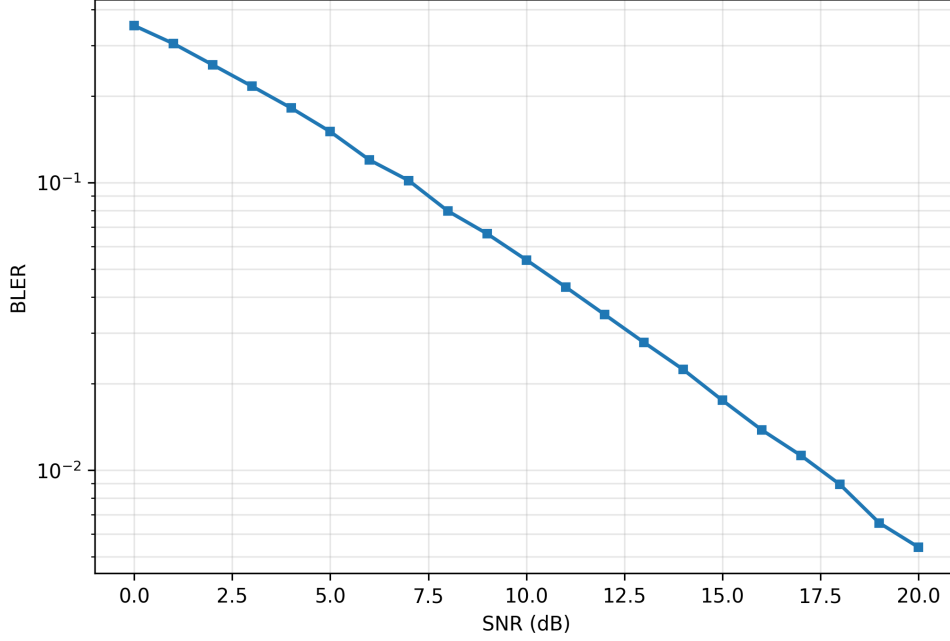


Figure 5.9: BLER versus SNR performance of the custom quantum encoder.

affected by noise and the symbol separation becomes more difficult. As the SNR increases, the SER decreases steadily, and after approximately 4 dB the improvement becomes much sharper. At higher SNR values, the HAE curve remains very close to both the learned and algorithmic references, reaching very low SER values. This indicates that the quantum encoder can maintain reliable ID performance while achieving a more parameter-efficient representation.

5.5 Conclusion

This thesis investigated the use of hybrid quantum-classical ML models in E2E wireless communication systems, with particular emphasis on parameter efficiency and communication performance. The main objective was to examine whether HAE architectures can reduce the number of trainable parameters while maintaining performance comparable to that of classical autoencoder-based systems.

To address this objective, the work first studied the reference HAE and CAE architectures under linear wireless channel conditions, including Rayleigh, Rician, and 3GPP fading models. The obtained results confirmed the main findings reported in the reference study, showing that the HAE can achieve BLER performance close to the CAE while requiring significantly fewer trainable parameters. This supports the view that hybrid quantum-classical learning can provide a more compact alternative to purely classical E2E communication models. The study then explored whether the number of trainable parameters in the quantum encoder could be reduced

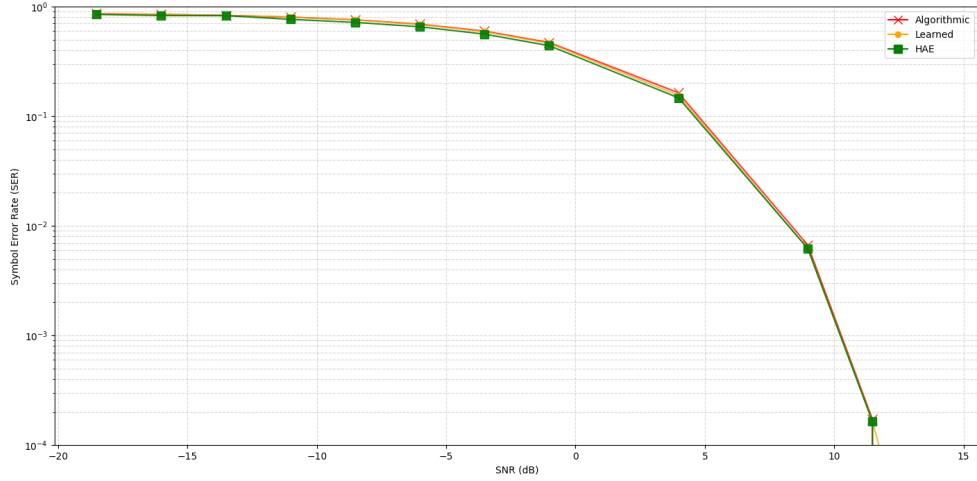


Figure 5.10: SER versus SNR comparison for the UR-SWIPT ID task, including the algorithmic reference, the learned classical model, and the HAE implementation.

even further. The results showed that the P18 variant preserves performance very close to that of the original HAE, while the more aggressive P12 reduction leads to a more visible degradation. Therefore, the results indicate that meaningful parameter compression is possible, although excessive reduction may affect reliability.

In addition, a new custom quantum encoder was designed and evaluated. Its training behavior and BLER performance under the Rayleigh channel showed that the proposed encoder follows a performance trend close to that of the reference (7, 4) HAE implementation. Although the architecture differs from the original model, the obtained results demonstrate that competitive performance can still be achieved through a different quantum encoder design. Finally, the hybrid approach was also examined in a non-linear wireless communication setting through the UR-SWIPT framework. The obtained results showed that the HAE remains competitive in the information decoding task while still providing reduced parameter complexity compared to the CAE. This suggests that the potential of hybrid quantum-classical learning extends beyond linear channel models and can also be relevant in more challenging nonlinear communication scenarios.

Overall, the results of this thesis show that hybrid quantum-classical autoencoders constitute a promising direction for parameter-efficient wireless communication system design. While quantum simulation still limits practical computational gains on classical hardware, the achieved reduction in trainable parameters, together with the competitive BLER and SER performance, highlights the value of hybrid models as compact and effective alternatives to classical architectures. Future work may further investigate improved quantum encoder designs, alternative embedding and measurement strategies, and more scalable implementations under broader com-

munication settings.

A Appendix

A.1 HPC Workflow and Experimental Infrastructure

Advanced ML and QML models are not typically trained or deployed using standard computing environments. Due to the high complexity of NN architectures and the continuously increasing number of trainable parameters, such models require substantial computational resources and carefully controlled random seeds to ensure reproducibility. For this reason, HPC infrastructure plays a critical role in the training and evaluation process.

A significant part of this work was dedicated to learning how to properly configure and utilize the HPC facilities of the University of Cyprus. Running experiments on the HPC requires the installation and configuration of all necessary software backends and libraries that are supported by the system administration. Once the computational environment is properly defined, experiments are executed through a SLURM job submission script, which specifies the requested computational resources such as CPUs, memory, and execution time.

After developing and validating the training scripts locally using conventional development tools (e.g., PyCharm), the codebase is version-controlled and transferred to the HPC environment via GitHub. From there, the experiments are executed exclusively through batch jobs. This workflow and the selected simulation backend are also illustrated in Fig. A.1. It is important to note that the HPC environment does not support real-time plotting or graphical visualization. Consequently, all relevant metrics and intermediate results must be explicitly saved during each run. These saved outputs are later processed and visualized using separate plotting scripts in compatible local environments (e.g., PyCharm or MATLAB).

A.2 Simulator Backend Choice for HPC Execution

Among the available Lightning devices, `lightning.gpu` represents the most suitable option for the present application due to its ability to exploit parallel execution on GPUs, which is particularly advantageous for training quantum autoencoders with repeated circuit evaluations. However, this backend was not used in this work, as access to GPU-enabled Lightning devices was not available through the employed HPC infrastructure. The `lightning.kokkos` device targets scalable execution on heterogeneous HPC systems and is primarily intended for simulations involving larger numbers of qubits or workloads that benefit from extensive CPU parallelism, which does not apply to the small-scale circuits considered in this thesis. Consequently, all experiments were conducted using the `lightning.qubit` device, which provides

an efficient CPU-based state-vector simulator well suited to batched evaluations of small parameterized circuits. Compared to the default `default.qubit` simulator, `lightning.qubit` offers significantly improved performance through optimized C++ implementations, making it the most appropriate and practical choice for the proposed HAE framework.

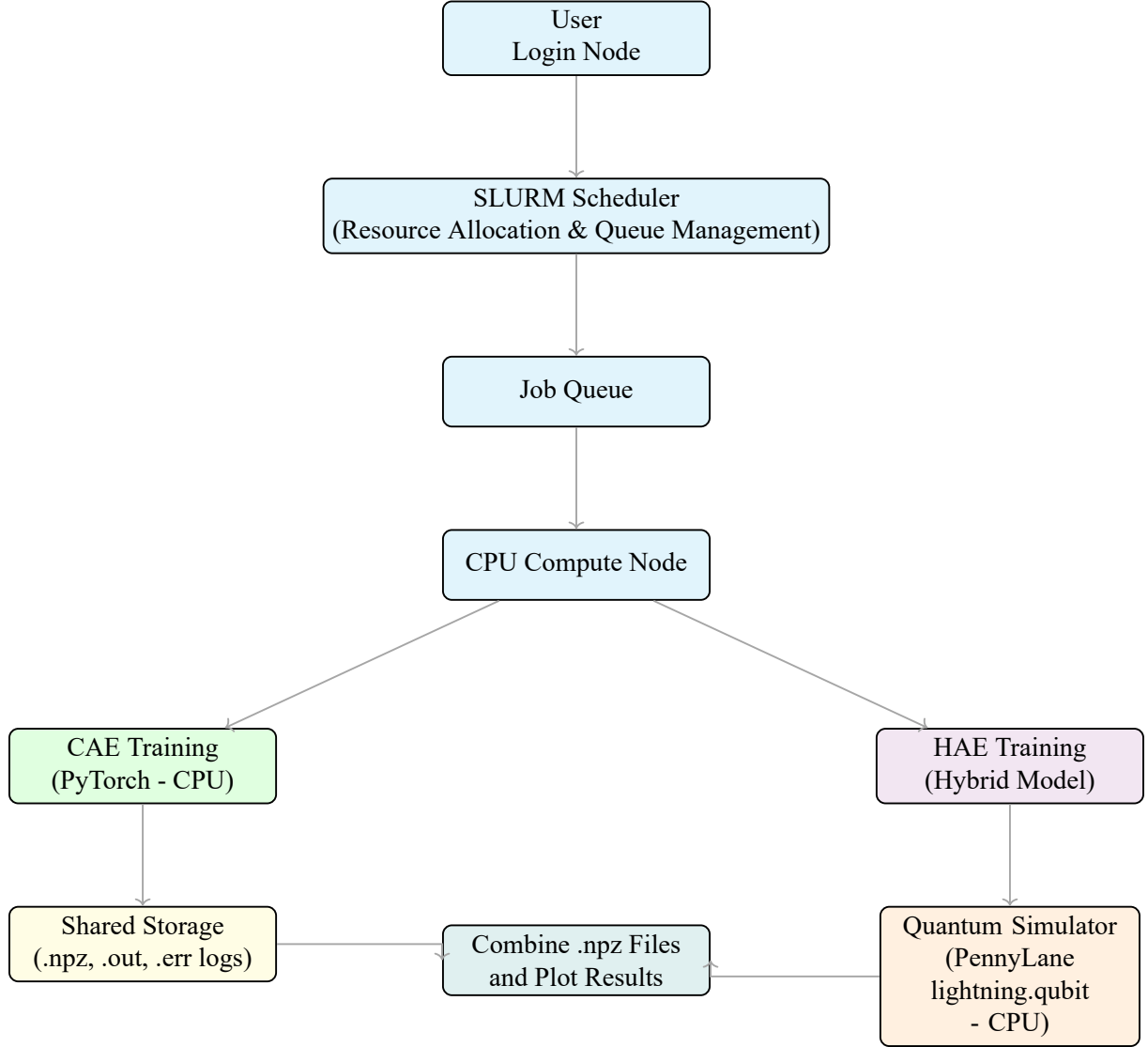


Figure A.1: HPC execution workflow using SLURM on CPU-based compute nodes for CAE and HAE experiments, including post-processing and visualization.

A.3 Creating a Quantum Device

In this section, we analyze how a quantum circuit is constructed within the PennyLane framework and examine the fundamental components that define its structure and operation.

In PennyLane, a quantum circuit must be associated with a quantum device, which specifies the

backend on which the circuit is executed. PennyLane provides multiple device implementations. In this work, we focus on noiseless qubit-based simulators.

When defining a device, the `wires` argument specifies the number of qubits in the quantum system. Each wire corresponds to a single qubit and determines how quantum operations are applied within the circuit.

An example device definition is given by:

```
dev = qml.device("lightning.qubit", wires=3)
```

This corresponds to a three-qubit quantum system. The ordering and labeling of wires can be adapted according to the circuit design.

To associate a quantum circuit with a device, PennyLane introduces the concept of a *QNode*. A QNode binds a quantum function to a specific device using the decorator `@qml.qnode`. The decorator is placed above the quantum function definition and enables automatic differentiation.

A.4 Additional Results

In this section, we provide additional results for the Hybrid UR-SWIPT model in the case $M = 4$. These results are not included in the main body of the thesis, since the main focus of this work is the study of parameter efficiency between hybrid and classical schemes, whereas in this particular setting the hybrid model does not offer a parameter-count advantage. Nevertheless, the obtained training results were very encouraging and indicated promising UR-SWIPT performance.

Table A.1 presents the trainable parameter counts of the hybrid scheme in comparison with the classical baseline. As discussed earlier, in this particular setting the quantum-based model does not use fewer trainable parameters than the classical one.

Table A.1: Trainable parameter comparison between the CAE and HAE models in the UR-SWIPT setting for $M = 4$.

Model	Encoder Parameters	Decoder Parameters	Total Parameters
CAE	25	28	53
HAE	32	28	60

We now turn to the detection performance of the $M = 4$ Hybrid UR-SWIPT model. As shown in Fig. A.2, the proposed HAE closely follows the performance of the learned classical model across the entire examined SNR range. At low SNR values, all schemes exhibit high SER, which is expected under strong noise conditions. As the SNR increases, however, the HAE curve

decreases almost identically to the learned classical curve and becomes clearly more favorable than the algorithmic reference in the moderate-to-high SNR region. In particular, around the transition region of the plot, the hybrid model preserves the steep SER decay of the learned solution, showing that the quantum-based encoder is able to capture the same effective decision structure even in the nonlinear UR-SWIPT setting.

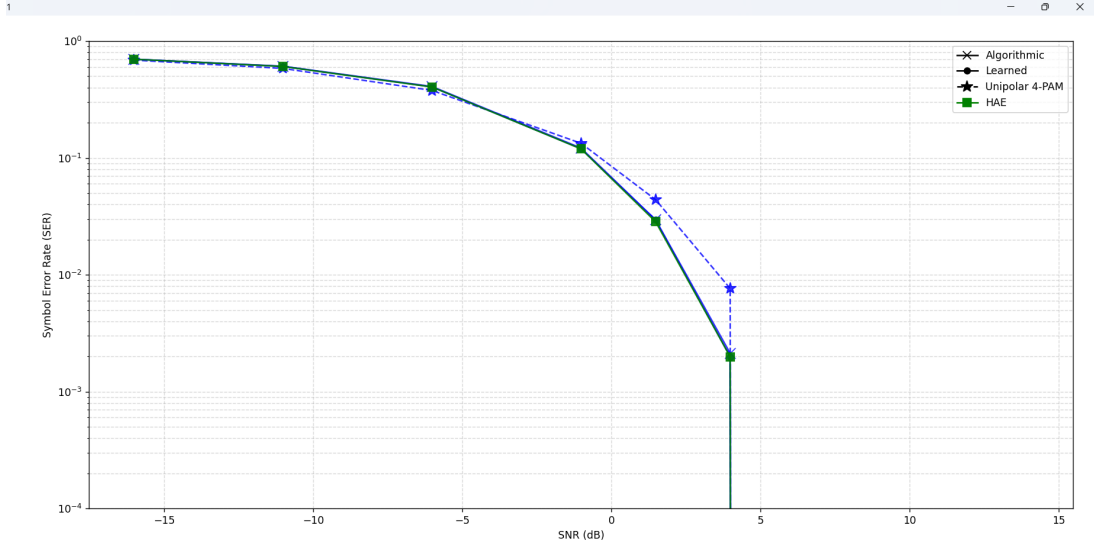


Figure A.2: SER performance of the Hybrid UR-SWIPT model for the $M = 4$ setting.

Additional insight is provided by the learned constellation shown in Fig. A.3. The figure compares the transmitted amplitude levels of the learned classical model, the algorithmic reference, and the HAE for several transmit-power values P_t . The most important observation is that the HAE points remain very close to the learned classical constellation across the full power range, while also following the same overall trend as the algorithmic design. At low P_t , the constellation points are compressed near the origin due to the limited available transmit power, whereas at higher P_t the four levels become more clearly separated. This behavior is consistent with the SER results of Fig. A.2, since better constellation separation at higher power leads to more reliable detection after the nonlinear rectifying receiver.

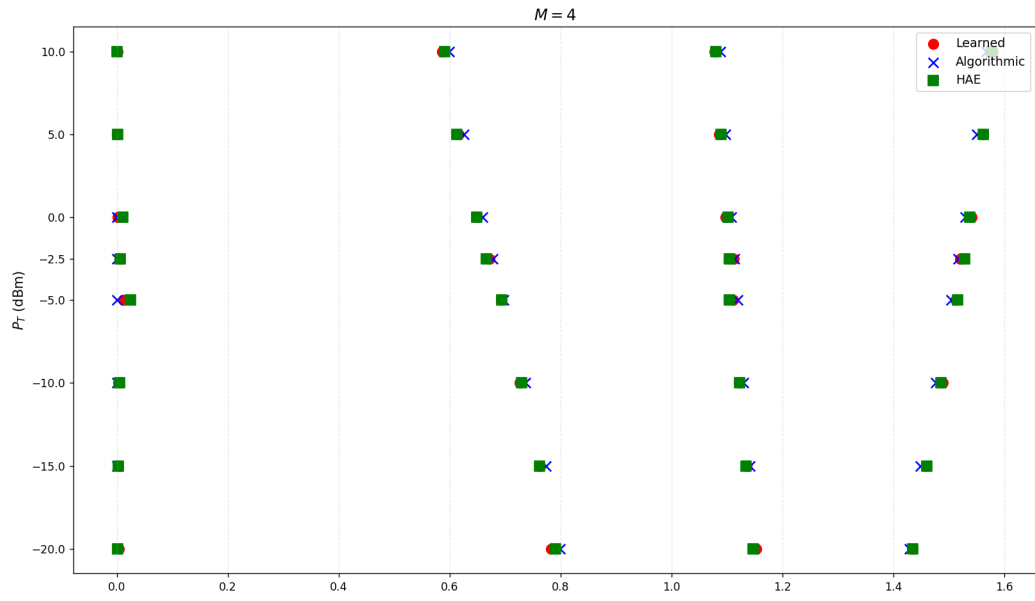


Figure A.3: Learned constellation of the Hybrid UR-SWIPT model for the $M = 4$ setting.

BIBLIOGRAPHY

- [1] T. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Transactions on Cognitive Communications and Networking*, vol. 3, no. 4, pp. 563–575, Dec. 2017.
- [2] H. Y. S, B. T. S, G. R. Sowkhya, R. K. K. N, and V. Karjigi, "Wireless communication using autoencoder," *International Journal / Conference (if applicable)*, 2020.
- [3] B. Zhang, G. Zheng, and N. V. Huynh, "A hybrid quantum-classical autoencoder framework for end-to-end communication systems," *IEEE Wireless Communications Letters*, vol. 14, no. 3, pp. 806–810, Mar. 2025.
- [4] R. P. Feynman, "Simulating physics with computers," *International Journal of Theoretical Physics*, vol. 21, no. 6, pp. 467–488, 1982.
- [5] D. Deutsch and R. Jozsa, "Rapid solution of problems by quantum computation," *Proceedings of the Royal Society A*, vol. 439, pp. 553–558, 1992.
- [6] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.
- [7] L. K. Grover, "A fast quantum mechanical algorithm for database search," *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pp. 212–219, 1996.
- [8] J. Preskill, "Quantum computing in the nisq era and beyond," *Quantum*, vol. 2, p. 79, 2018.
- [9] A. Asadi, A. Dusko, V. Michaud-Rioux, C.-Y. Park, I. Chochas, S. Shu, T. Vincent, and L. J. O'Riordan, "Hybrid quantum programming with pennylane lightning on hpc platforms," *Xanadu Technical Report*, 2024.
- [10] C. Gregg and K. Hazelwood, "Where is the data? why you cannot debate cpu vs. gpu performance without the answer," in *2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2011, pp. 134–144.
- [11] S. Y.-C. Chen, H.-H. Tseng, H.-Y. Lin, and S. Yoo, "Learning to program quantum measurements for machine learning," *arXiv preprint arXiv:2505.13525*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.13525>
- [12] T. Mavrovoltsos, E. Faddoul, Z. B. Ashraf, C. Psomas, B. Smida, and I. Krikidis, "Autoencoder-based constellation learning for unified swipt receivers," *IEEE*, 2025, under review / preprint.

- [13] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Prentice Hall, 2009.
- [14] C. E. Shannon, “A mathematical theory of communication,” *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 5, no. 1, pp. 3–55, 2001.
- [15] A. Goldsmith, *Wireless Communications*. Cambridge University Press, 2005.
- [16] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. McGraw-Hill, 2007.
- [17] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [18] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- [19] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.
- [20] H. Nikolić, “Epr before epr: a 1930 einstein–bohr thought experiment revisited,” *European Journal of Physics*, vol. 33, no. 5, pp. 1089–1097, 2012.
- [21] D. G. Bussandri and P. W. Lamberti, “Quantum metrology and quantum correlations,” in *Handbook of Statistics*, A. Plastino, A. S. Srinivasa Rao, and C. Rao, Eds. Elsevier, 2021, vol. 45, pp. 149–160.
- [22] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini, “Parameterized quantum circuits as machine learning models,” *Quantum Science and Technology*, vol. 4, no. 4, p. 043001, 2019.
- [23] IBM, “What is high performance computing (hpc)?” 2023, accessed: 2026. [Online]. Available: <https://www.ibm.com/topics/high-performance-computing>
- [24] A. de Lima Luiz, S. V. Kurlekar, and M. Georges, “Scalable engine and the performance of different llm models in a slurm-based hpc architecture,” *arXiv preprint arXiv:2508.17814*, 2025. [Online]. Available: <https://arxiv.org/abs/2508.17814>
- [25] PennyLane, “Circuit ansatz,” https://pennylane.ai/qml/glossary/circuit_ansatz, 2026, accessed: 2026-05-07.
- [26] R. Wiersema, C. Zhou, Y. de Sereville, J. F. Carrasquilla, Y. B. Kim, and H. Yuen, “Exploring entanglement and optimization within the hamiltonian variational ansatz,” *PRX Quantum*, vol. 1, no. 2, p. 020319, 2020. [Online]. Available: <https://doi.org/10.1103/PRXQuantum.1.020319>
- [27] M. A. Khan, M. N. Aman, and B. Sikdar, “Beyond bits: A review of quantum embedding techniques for efficient information processing,” *IEEE Access*, vol. 12, pp. 45 793–45 826, 2024.
- [28] 3GPP, “Spatial channel model for Multiple Input Multiple Output (MIMO) simulations,” 3rd Generation Partnership Project (3GPP), Technical Report TR 25.996, 2025, release 19.